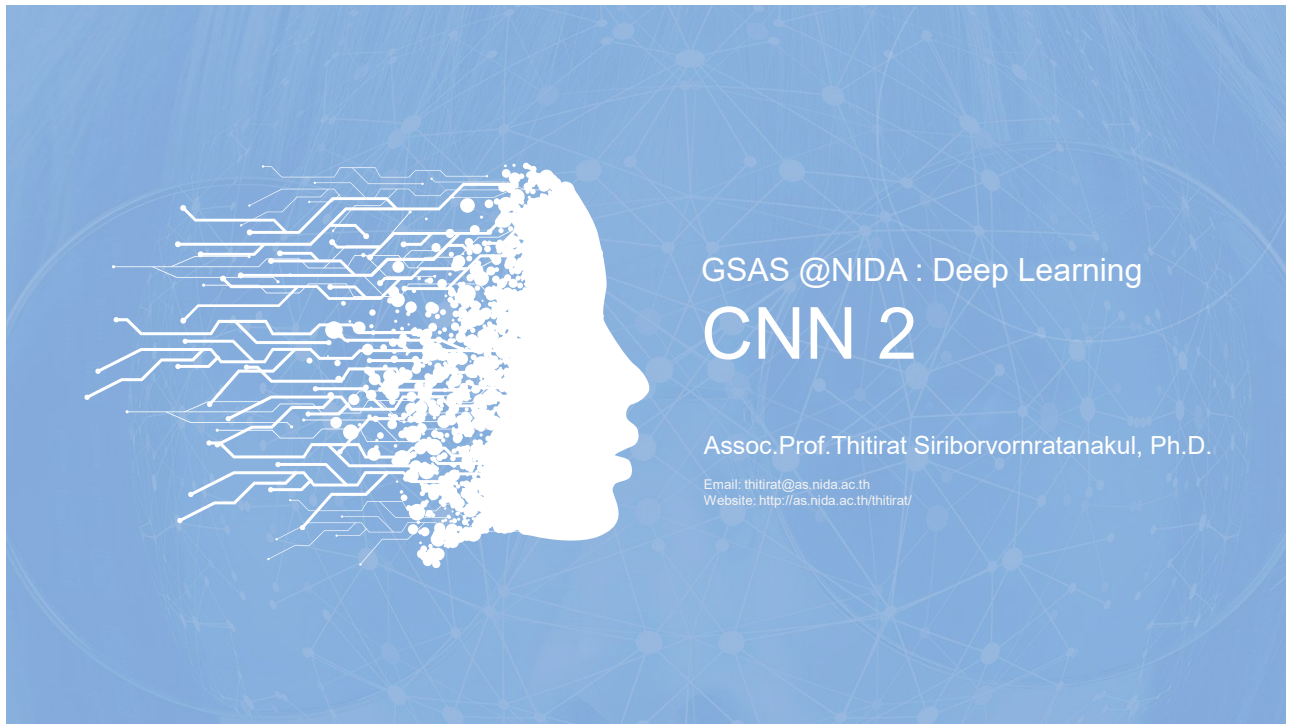




1

# OUTLINE

## 01 State-of-the-art Classifiers

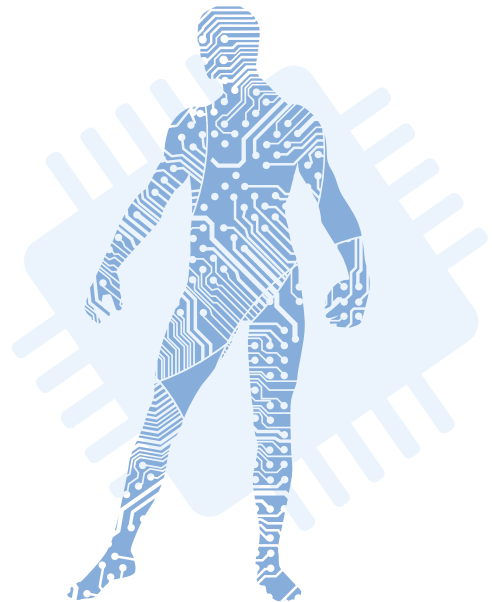
Dig in CNN's components through famous state-of-the-art CNN-based image classifiers

## 02 Foundation Models

Jump start our CNNs with the big foundation models and their pre-trained weights

## 03 Demystify CNNs

Make CNNs more transparent and explainable



2



# SOTA Classifiers

Dig in CNN's components through famous state-of-the-art CNN-based image classifiers

3

## THE OLD HISTORY

- **1943:** The first **neural network** was proposed by McCulloch and Pitts.
- **1955:** John McCarthy first coined the term **Artificial Intelligence**.
- **1959:** Samuel coined and popularized the term **Machine Learning**.

- **1982:** Birth of **Hopfield Network**
- **1986:** Birth of **backpropagation**
- **1986:** Birth of **RNN** and **AE**
- **1989:** Birth of **CNN**
- **1997:** Birth of **LSTM**

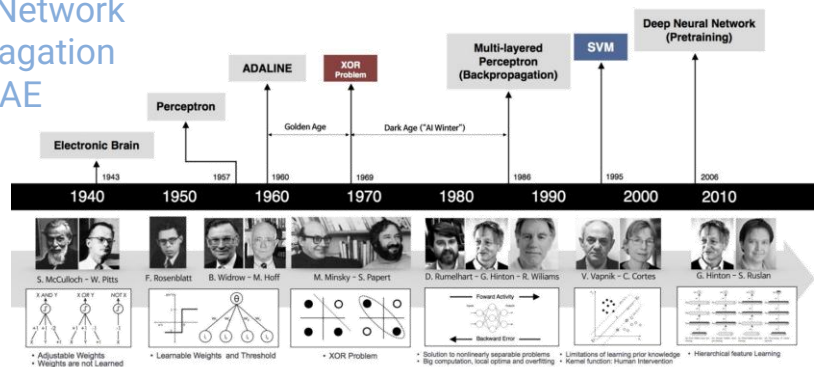


Image credit:  
[https://beamandrew.github.io/deeplearning/2017/02/23/deep\\_learning\\_101\\_part1.html](https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html)

4

# AI THE MODERN HISTORY

- **2006:** Hinton's unsupervised layer-wise **pretraining** technique reignited the DL term in the research community



- **2012:** ImageNet evolution—**AlexNet**

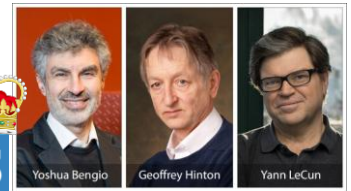


- **2014:** Birth of **GRU**, **VAE**, and **GAN**
- **2015:** Birth of **diffusion model**

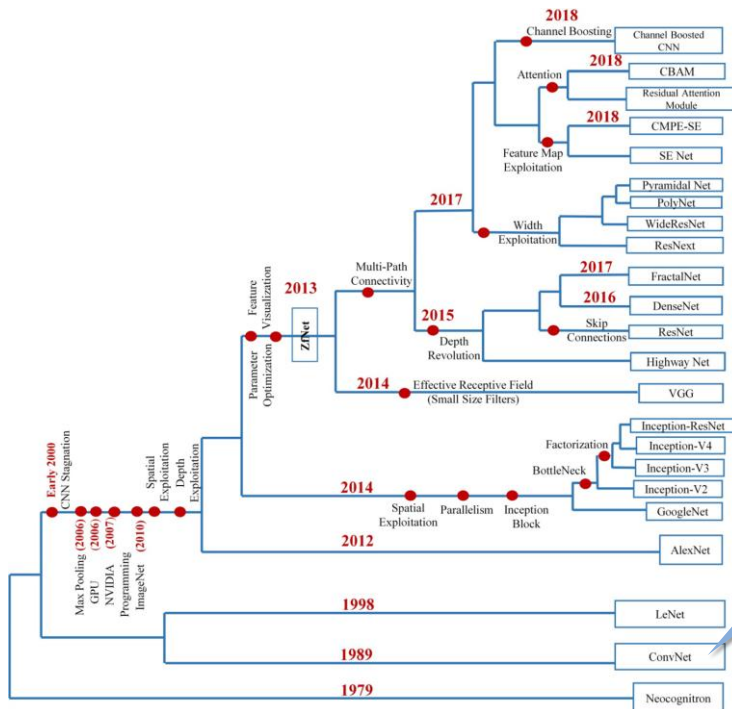
- **2017:** The **DeepFake** viral
- **2017:** Birth of the groundbreaking **Transformer**



- **2018:** NLP's ImageNet moment—**BERT**
- **2018:** ACM Turing Award to Deep NN



5



The first multilayered CNN named ConvNet whose origin rooted in Fukushima's Neocognitron

LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, 1(4), pp. 541-551, 1989.  
<http://yann.lecun.com/exdb/publis/pdf/lecun-89e.pdf>

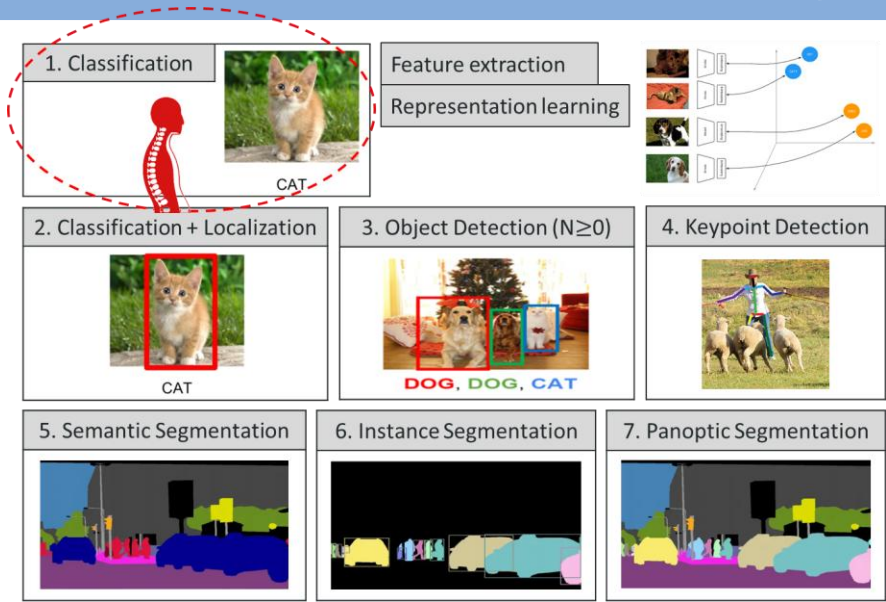
Image credit:

"A survey of the recent architectures of deep convolutional neural networks," *Artificial Intelligence Review*, 53, pp. 5455-5516, 2020.  
<https://link.springer.com/article/10.1007/s10462-020-09825-6>

6



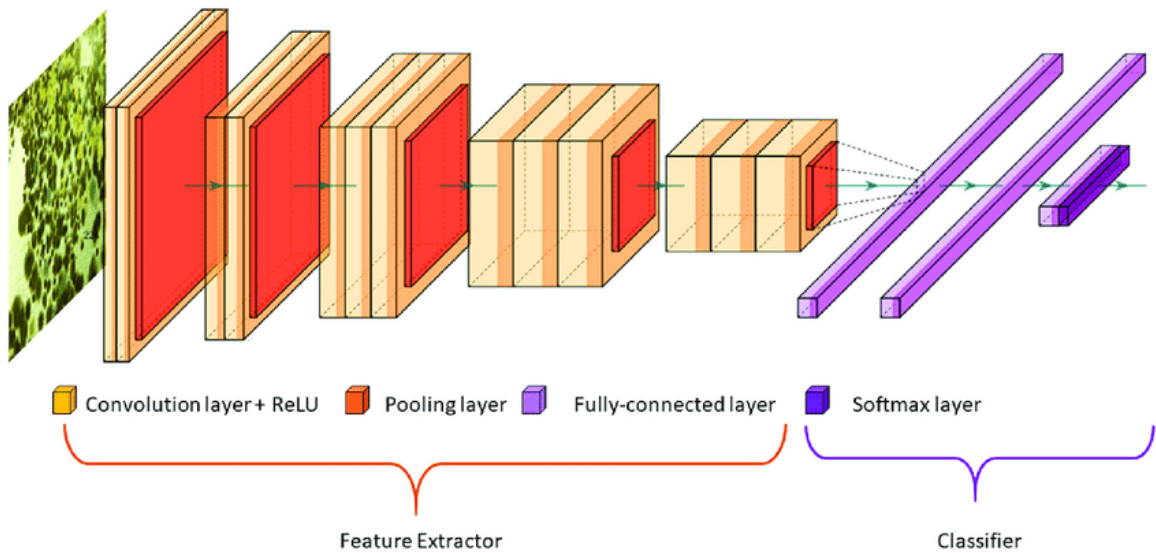
# Basic DL tasks on 2D Image



7



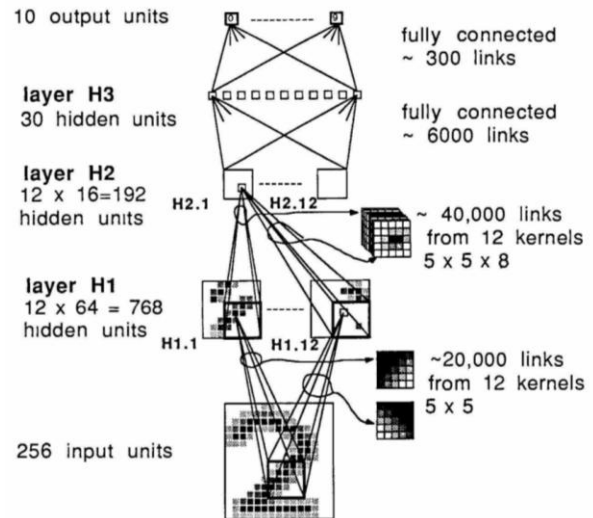
# CNN for Image Classification



8

# ConvNet (1989)

- The first multilayered CNN named **ConvNet** whose origin rooted in Fukushima's Neocognitron

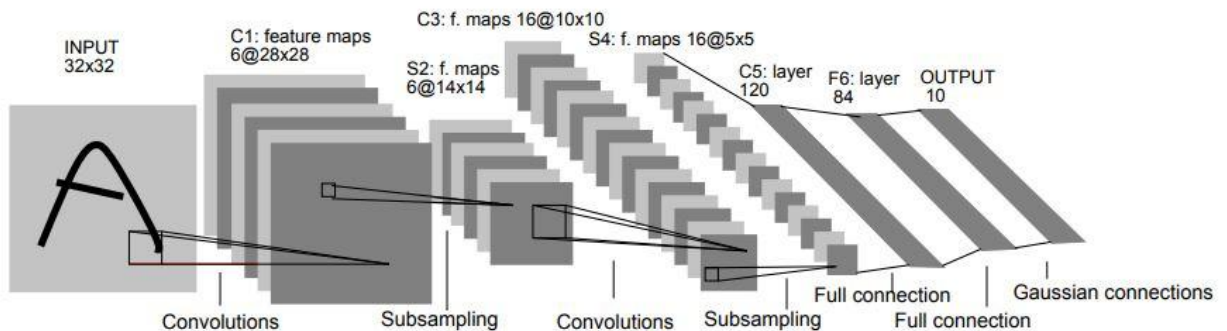


LeCun et al., "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, 1(4), pp. 541-551, 1989. <http://yann.lecun.com/exdb/publis/pdf/lecun-89e.pdf>

9

# LeNet-5 (1998)

- LeNet-5** for MNIST dataset by Yann LeCun et al.



Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner, "Gradient-based learning applied to document recognition," In *Procs. of the IEEE*, 1998 <http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>

10

# AlexNet (ImageNet 2012, NIPS 2012)

## ImageNet Classification with Deep Convolutional Neural Networks

**Alex Krizhevsky**  
University of Toronto  
kriz@cs.utoronto.ca

**Ilya Sutskever**  
University of Toronto  
ilya@cs.utoronto.ca

**Geoffrey E. Hinton**  
University of Toronto  
hinton@cs.utoronto.ca

### Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet LSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called "dropout" that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

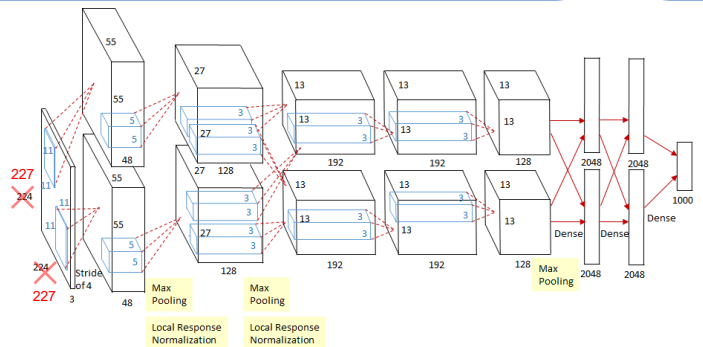


Image credit: <https://www.zdnet.com/article/alexnet-the-ai-model-that-started-it-all-released-in-source-code-form-for-all-to-download/>

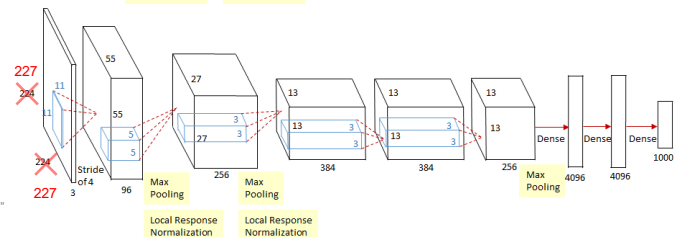
11

# AlexNet (ImageNet 2012, NIPS 2012)

- **AlexNet** (2 GPUs): 8 layers
  - Winner of ImageNet 2012
  - Trained for 90 epochs in 6 days using two NVIDIA GeForce GTX 580 GPUs (each with 3GB memory)
  - Two regularization techniques: dropout and data augmentation



- **CaffeNet** (1 GPU):



A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," In Proc. of NeurIPS, 2012 <https://papers.nips.cc/paper/2012/hash/c39862d3b9d6b76c8436e924a68c45b-Abstract.html>

12

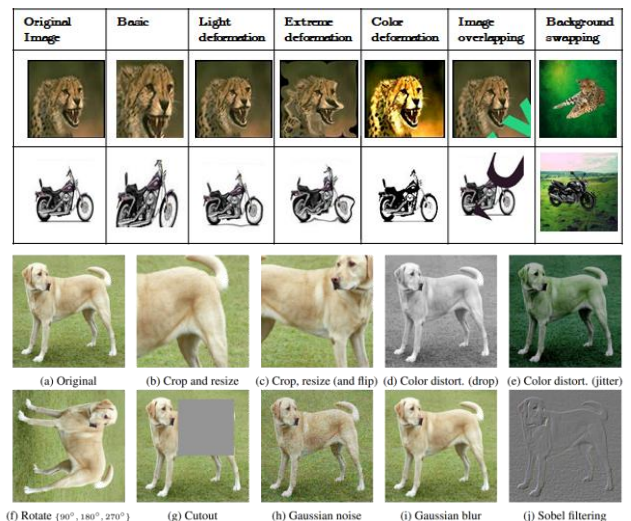


13

## (2D Image) Data Augmentation

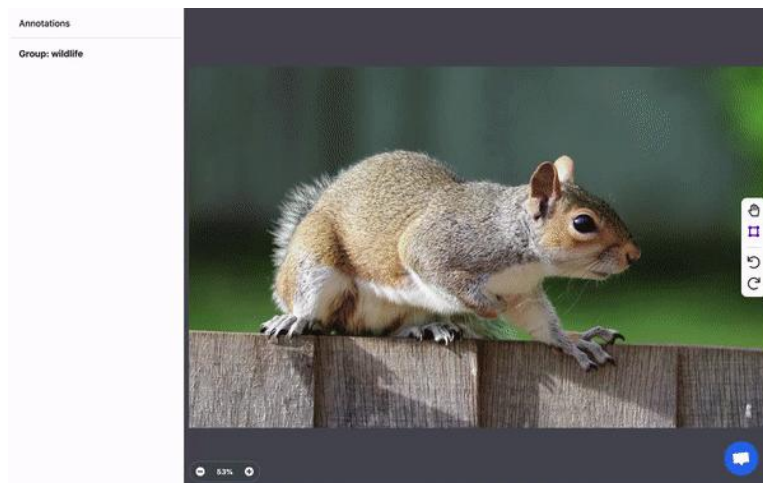
- Artificially increase the size of the training set by generating many realistic variants of each training instance
- Data augmentation** reduces overfitting, making it a regularization technique.

**Test-Time Augmentation (TTA)**—augmenting the test image and averaging the predictions—is a common technique in competitions to squeeze out the last 0.5–2% during inference.



14

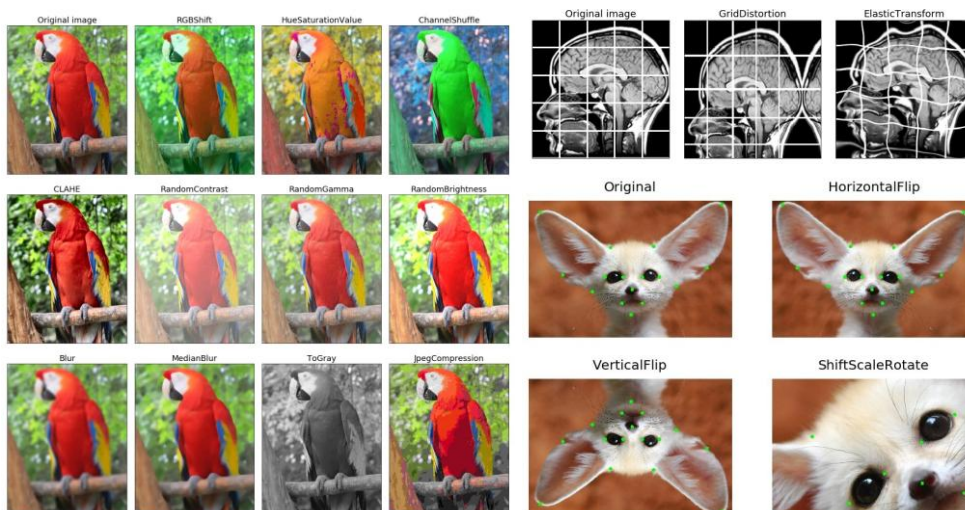
# (2D Image) Data Augmentation



<https://roboflow.com>

15

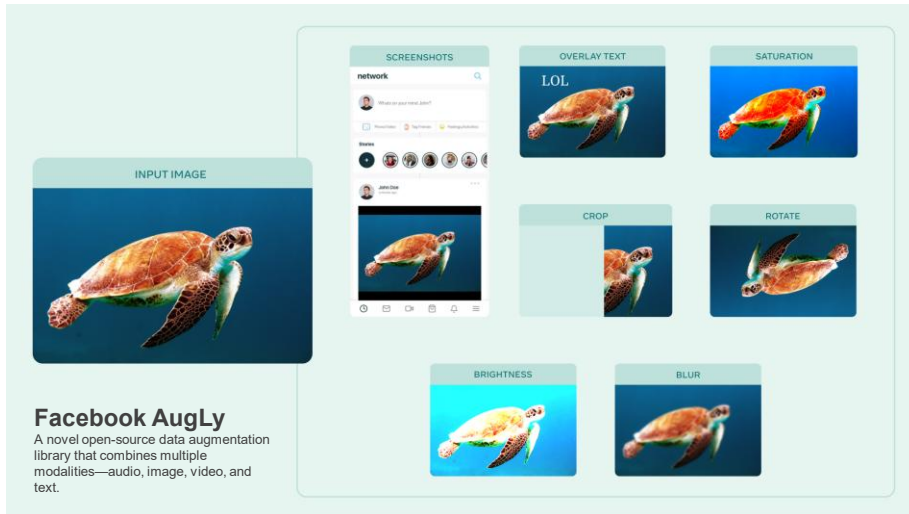
# (2D Image) Data Augmentation



<https://github.com/albumentations-team/albumentations>

16

# (2D Image) Data Augmentation



17JUN2021: <https://ai.facebook.com/blog/augly-a-new-data-augmentation-library-to-help-build-more-robust-ai-models>

17

## (2D Image) Data Augmentation

- **On-the-fly Transformation:** Augmentations are typically applied to the data every time a batch is generated during training, meaning the original dataset remains unchanged, and each epoch sees slightly different variations.
- **Composability:** Transformations can be chained together into a single pipeline using `torchvision.transforms.v2.Compose()`.
  - For some transformations with two alternative usages, like `v2.Resize()` and `v2.functional.resize()`, the first is a transform class used as part of a `Compose` pipeline for data loading, while the latter is a stateless function that offers fine-grained control over the transformation process.
- **v1 vs. v2 Transforms:** The `torchvision.transforms.v2` module is recommended over the legacy v1 because it is faster, supports more functionalities, and can apply the same random transformations to multiple inputs (e.g., image and segmentation mask).

### Transforming and augmenting images

Start here

+ Supported input types and conventions

V1 or V2? Which one should I use?

Performance considerations

Transform classes, functionals, and kernels

Torchscript support

- V2 API reference - Recommended

+ Geometry

+ Color

+ Composition

+ Miscellaneous

+ Conversion

Auto-Augmentation

CutMix - MixUp

- Developer tools

+ V1 API Reference



<https://docs.pytorch.org/vision/0.21/transforms.html#>

18

# (2D Image) Data Augmentation

## Geometry

### Resizing

|                                                               |                                                                                                                                          |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>v2.Resize(size[, interpolation, max_size, ...])</code>  | Resize the input to the given size.                                                                                                      |
| <code>v2.ScaleJitter(target_size[, scale_range, ...])</code>  | Perform Large Scale Jitter on the input according to "Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation". |
| <code>v2.RandomShortestSize(min_size[, max_size, ...])</code> | Randomly resize the input.                                                                                                               |
| <code>v2.RandomSize(min_size, max_size[, ...])</code>         | Randomly resize the input.                                                                                                               |

### Cropping

|                                                           |                                                                                                                                            |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>v2.RandomCrop(size[, padding, ...])</code>          | Crop the input at a random location.                                                                                                       |
| <code>v2.RandomSizeCrop(size[, scale, ratio, ...])</code> | Crop a random portion of the input and resize it to a given size.                                                                          |
| <code>v2.RandomIoUCrop(min_scale, max_scale, ...)</code>  | Random IoU crop transformation from "SSD: Single Shot MultiBox Detector".                                                                  |
| <code>v2.CenterCrop(size)</code>                          | Crop the input at the center.                                                                                                              |
| <code>v2.FiveCrop(size)</code>                            | Crop the image or video into four corners and the central crop.                                                                            |
| <code>v2.TenCrop(size[, vertical_flip])</code>            | Crop the image or video into four corners and the central crop plus the flipped version of these (horizontal flipping is used by default). |

## Others

|                                                                |                                                                                    |
|----------------------------------------------------------------|------------------------------------------------------------------------------------|
| <code>v2.RandomHorizontalFlip([p])</code>                      | Horizontally flip the input with a given probability.                              |
| <code>v2.RandomVerticalFlip([p])</code>                        | Vertically flip the input with a given probability.                                |
| <code>v2.Pad(padding[, fill, padding_mode])</code>             | Pad the input on all sides with the given "pad" value.                             |
| <code>v2.RandomZoomOut([fill, side_range, p])</code>           | "Zoom out" transformation from "SSD: Single Shot MultiBox Detector".               |
| <code>v2.RandomRotation(degrees[, interpolation, ...])</code>  | Rotate the input by angle.                                                         |
| <code>v2.RandomAffine(degrees[, translate, scale, ...])</code> | Random affine transformation the input keeping center invariant.                   |
| <code>v2.RandomPerspective([distortion_scale, p, ...])</code>  | Perform a random perspective transformation of the input with a given probability. |
| <code>v2.ElasticTransform([alpha, sigma, ...])</code>          | Transform the input with elastic transformations.                                  |



<https://docs.pytorch.org/vision/0.21/transforms.html#geometry>

19

## Scale Jittering Augmentation

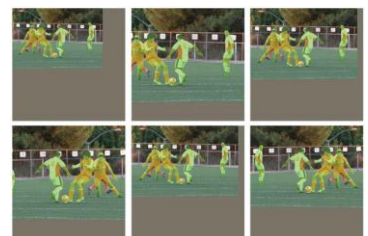
### • Scale jittering augmentation:

Available in `torchvision.transforms.v2.ScaleJitter(...)`

- Randomly resize and crop images
- If images are made smaller than their original size, then the images are padded with gray pixel values.
- **Standard scale jittering (SSJ)**: a resize range of 0.8 to 1.25 of the original image size
- **Large-scale jittering (LSJ)**: a resize range of 0.1 to 2.0 of the original image size

### • Research finding:

- **LSJ** yields significant performance improvements over the standard scale jittering used in most prior works.



(a) Standard Scale Jittering (SSJ)



(b) Large Scale Jittering (LSJ)



"Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation," CVPR 2021 (Oral) <https://arxiv.org/abs/2012.07177>, <https://github.com/conradry/copy-paste-aug>

20

# Copy-Paste Augmentation (for Instance Segmentation)



- **Copy-paste augmentation:** A simple strategy of randomly picking diverse objects of various scales and pasting them to new background images at random locations
- **Research finding:** This simple mechanism of pasting objects randomly is good enough and can provide solid gains on top of strong baselines.



"Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation," CVPR 2021 (Oral) <https://arxiv.org/abs/2012.07177>, <https://github.com/conradry/copy-paste-aug>

21

## (2D Image) Data Augmentation

Color

|                                                             |                                                                                      |                                                              |                                                                                                              |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>v2.ColorJitter([brightness, contrast, ...])</code>    | Randomly change the brightness, contrast, saturation and hue of an image or video.   | <code>v2.GaussianNoise([mean, sigma, clip])</code>           | Add gaussian noise to images or videos.                                                                      |
| <code>v2.RandomChannelPermutation()</code>                  | Randomly permute the channels of an image or video                                   | <code>v2.RandomInvert([p])</code>                            | Inverts the colors of the given image or video with a given probability.                                     |
| <code>v2.RandomPhotometricDistort([brightness, ...])</code> | Randomly distorts the image or video as used in SSD: Single Shot MultiBox Detector.  | <code>v2.RandomPosterize([bits, p])</code>                   | Posterize the image or video with a given probability by reducing the number of bits for each color channel. |
| <code>v2.Grayscale([num_output_channels])</code>            | Convert images or videos to grayscale.                                               | <code>v2.RandomSolarize([threshold, p])</code>               | Solarize the image or video with a given probability by inverting all pixel values above a threshold.        |
| <code>v2.RGB()</code>                                       | Convert images or videos to RGB (if they are already not RGB).                       | <code>v2.RandomAdjustSharpness([sharpness_factor, p])</code> | Adjust the sharpness of the image or video with a given probability.                                         |
| <code>v2.RandomGrayscale([p])</code>                        | Randomly convert image or videos to grayscale with a probability of p (default 0.1). | <code>v2.RandomAutocontrast([p])</code>                      | Autocontrast the pixels of the given image or video with a given probability.                                |
| <code>v2.GaussianBlur([kernel_size, sigma])</code>          | Blurs image with randomly chosen Gaussian blur kernel.                               | <code>v2.RandomEqualize([p])</code>                          | Equalize the histogram of the given image or video with a given probability.                                 |



<https://docs.pytorch.org/vision/0.21/transforms.html#color>

22

# (2D Image) Data Augmentation

## Composition

`v2.Compose(transforms)`

Composes several transforms together.

`v2.RandomApply(transforms[, p])`

Apply randomly a list of transformations with a given probability.

`v2.RandomChoice(transforms[, p])`

Apply single transformation randomly picked from a list.

`v2.RandomOrder(transforms)`

Apply a list of transformations in a random order.



<https://docs.pytorch.org/vision/0.21/transforms.html#composition>

23

# (2D Image) Data Augmentation

## Miscellaneous

`v2.LinearTransformation(...)`

Transform a tensor image or video with a square transformation matrix and a mean\_vector computed offline.

`v2.Normalize(mean, std[, inplace])`

Normalize a tensor image or video with mean and standard deviation.

`v2.RandomErasing([p, scale, ratio, value, ...])`

Randomly select a rectangle region in the input image or video and erase its pixels.

`v2.Lambda(lambd, *types)`

Apply a user-defined function as a transform.

`v2.SanitizeBoundingBoxes([min_size, ...])`

Remove degenerate/invalid bounding boxes and their corresponding labels and masks.

`v2.ClampBoundingBoxes()`

Clamp bounding boxes to their corresponding image dimensions.

`v2.UniformTemporalSubsample(num_samples)`

Uniformly subsample `num_samples` indices from the temporal dimension of the video.

`v2.JPEG(quality)`

Apply JPEG compression and decompression to the given images.



<https://docs.pytorch.org/vision/0.21/transforms.html#miscellaneous>

24

# (2D Image) Data Augmentation

## Conversion

### • NOTE

Beware, some of these conversion transforms below will scale the values while performing the conversion, while some may not do any scaling. By scaling, we mean e.g. that a `uint8`  $\rightarrow$  `float32` would map the `[0, 255]` range into `[0, 1]` (and vice-versa). See [Dtype and expected value range](#).

`v2.ToImage()`

Convert a tensor, ndarray, or PIL Image to **Image**; this does not scale values.

`v2.ToPILImage()`

Convert all TVTensors to pure tensors, removing associated metadata (if any).

`v2.PILToTensor()`

Convert a PIL Image to a tensor of the same type - this does not scale values.

`v2.ToPILImage(mode)`

Convert a tensor or an ndarray to PIL Image

`v2.ToDtype(dtype[, scale])`

Converts the input to a specific dtype, optionally scaling the values for images or videos.

`v2.ConvertBoundingBoxFormat(format)`

Convert bounding box coordinates to the given **format**, eg from "CXYWH" to "XYXY".



<https://docs.pytorch.org/vision/0.21/transforms.html#conversion>

25

# (2D Image) Data Augmentation

## Auto-Augmentation

**AutoAugment** is a common Data Augmentation technique that can improve the accuracy of Image Classification models. Though the data augmentation policies are directly linked to their trained dataset, empirical studies show that ImageNet policies provide significant improvements when applied to other datasets. In TorchVision we implemented 3 policies learned on the following datasets: ImageNet, CIFAR10 and SVHN. The new transform can be used standalone or mixed-and-matched with existing transforms:

`v2.AutoAugment([policy, interpolation, fill])`

AutoAugment data augmentation method based on "AutoAugment: Learning Augmentation Strategies from Data".

→ Automatically search for the best augmentation policy that the neural network yields the highest validation accuracy.  
→ CVPR 2019 <https://arxiv.org/abs/1805.09501>

`v2.RandAugment([num_ops, magnitude, ...])`

RandAugment data augmentation method based on "RandAugment: Practical automated data augmentation with a reduced search space".

→ Automated augmentation policies with a reduced search space.  
→ NIPS 2020 <https://arxiv.org/abs/1909.13719>

`v2.TrivialAugmentWide([num_magnitude_bins, ...])`

Dataset-independent data-augmentation with TrivialAugment Wide, as described in "TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation".

→ A highly efficient, parameter-free image data augmentation technique that achieves SOTA results without needing to tune hyperparameters. It works by randomly selecting a single transformation and applying it to each image.  
→ ICCV 2021 oral <https://arxiv.org/abs/2103.10158>

`v2.AugMix([severity, mixture_width, ...])`

AugMix data augmentation method based on "AugMix: A Simple Data Processing Method to Improve Robustness and Uncertainty".

→ Mix many augmentations w/o degrading or drifting off the data manifold.  
→ ICLR 2020 <https://arxiv.org/abs/1912.02781>



<https://docs.pytorch.org/vision/0.21/transforms.html#auto-augmentation>

26

# (2D Image) Data Augmentation

## CutMix - MixUp

CutMix and MixUp are special transforms that are meant to be used on batches rather than on individual images, because they are combining pairs of images together. These can be used after the dataloader (once the samples are batched), or part of a collation function. See [How to use CutMix and MixUp](#) for detailed usage examples.

`v2.CutMix(*[, alpha, num_classes, labels_getter])` Apply CutMix to the provided batch of images and labels.  
→ Patches are cut and pasted among training images, ICCV 2019 <https://arxiv.org/abs/1905.04899>

`v2.MixUp(*[, alpha, num_classes, labels_getter])` Apply MixUp to the provided batch of images and labels.  
→ MixUp augmentation technique for object detection, arXiv 2019 <https://arxiv.org/abs/1902.04103>

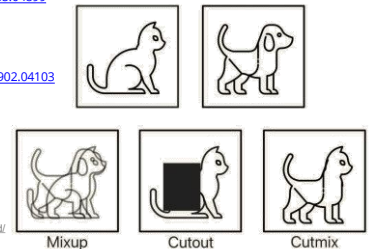


Image credit: 14APR2023 <https://towardsdatascience.com/cutout-mixup-and-cutmix-implementing-modern-image-augmentations-in-pytorch-a9d7db3074ad/>



<https://docs.pytorch.org/vision/0.21/transforms.html#cutmix-mixup>

27

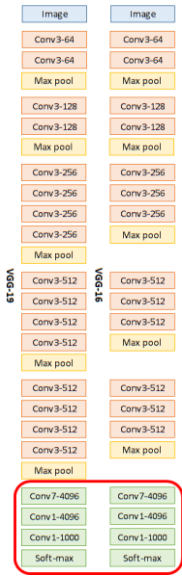
## EX1: (2D Image) Data augmentation

- Example of training a CNN model with image data augmentation



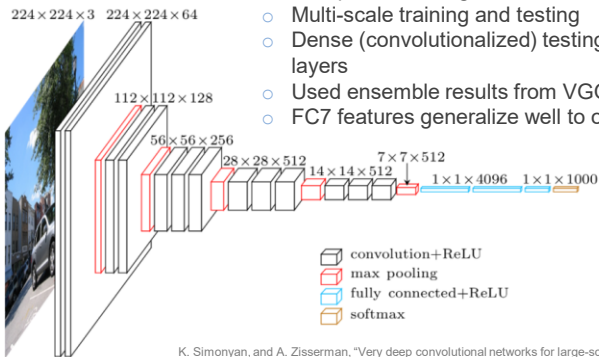
28

# A Oxford VGG (ImageNet 2014, ICLR 2015)



- **VGG16** (16 layers) and **VGG19** (19 layers): The 1<sup>st</sup> runner up in ImageNet 2014
- **VGG16** is one of the most popular CNN backends. Although VGG19 is a bit better in accuracy, it also consumes more memory.
- Trained 2-3 weeks using four NVIDIA Titan Black (6GB) GPUs

- **Tricks:**
  - Used a hierarchy of 3x3 convolutions instead of 5x5 and 11x11 convolutions; the spatial coverage was the same but the number of parameters was reduced
  - Multi-scale training and testing
  - Dense (convolutionalized) testing: during testing, replace FC layers with CNN layers
  - Used ensemble results from VGG16 and VGG19 for the best results
  - FC7 features generalize well to other tasks, suitable for transfer learning

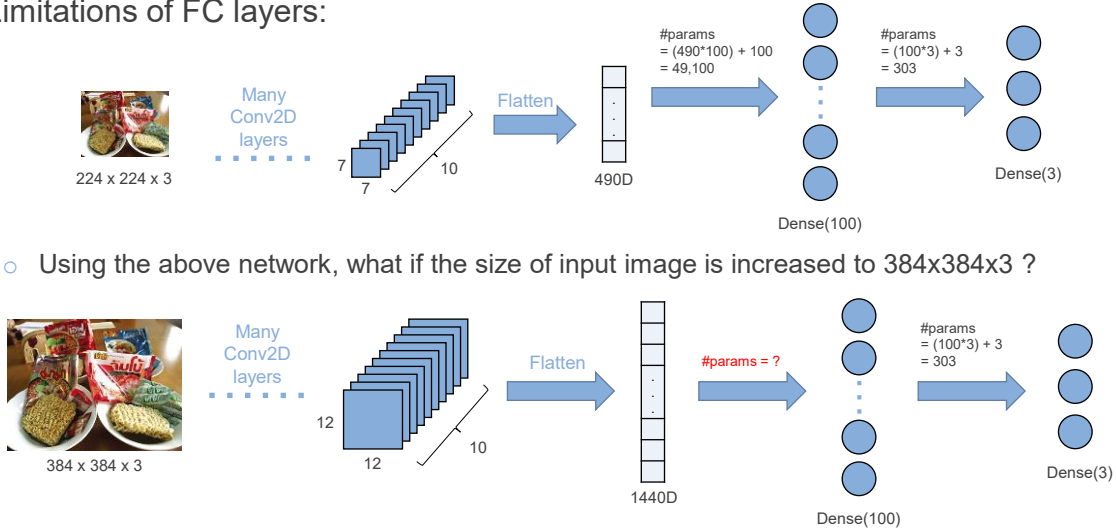


K. Simonyan, and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," In Proc. of ICLR, 2015 <https://arxiv.org/abs/1409.1556>

29

## FC layer ⇔ Conv layer

- Limitations of FC layers:



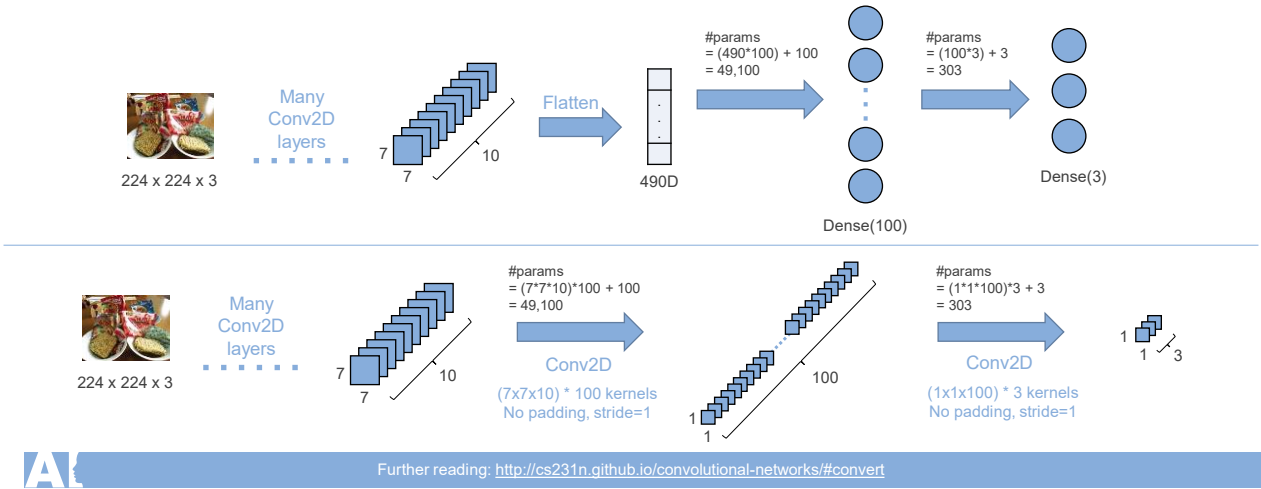
- Using the above network, what if the size of input image is increased to 384x384x3 ?

Further reading: <http://cs231n.github.io/convolutional-networks/#convert>

30

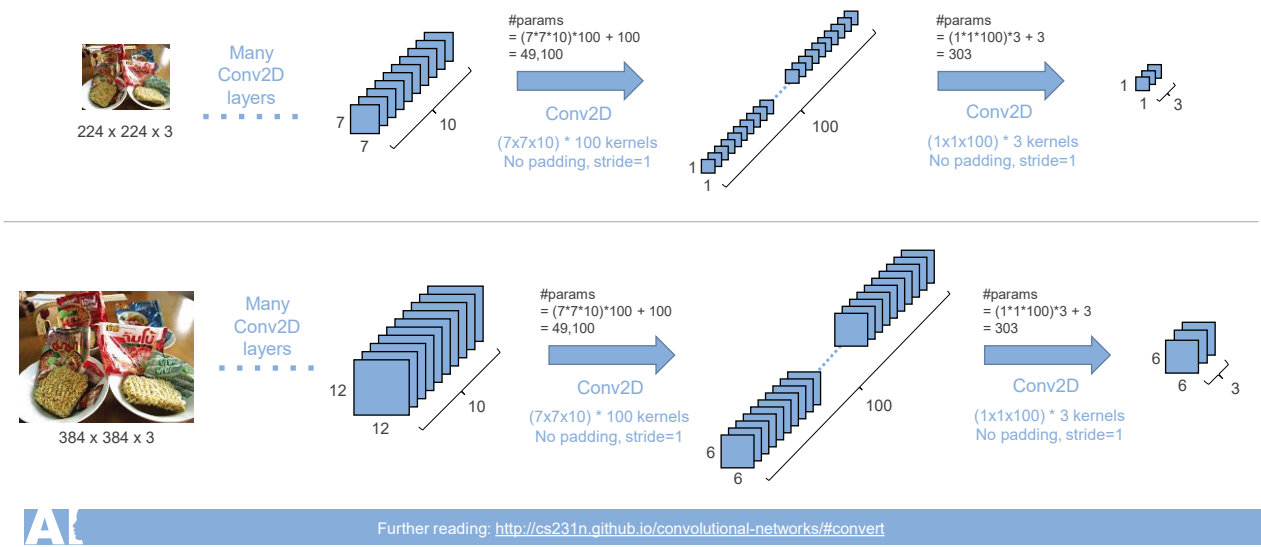
# FC layer ⇔ Conv layer

- **Deconvolutionalize**: convert from a Conv layer to an equivalent FC layer
- **Convolutionalize**: convert from a FC layer to an equivalent Conv layer



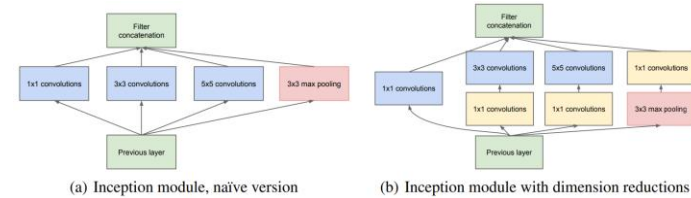
31

# FC layer ⇔ Conv layer

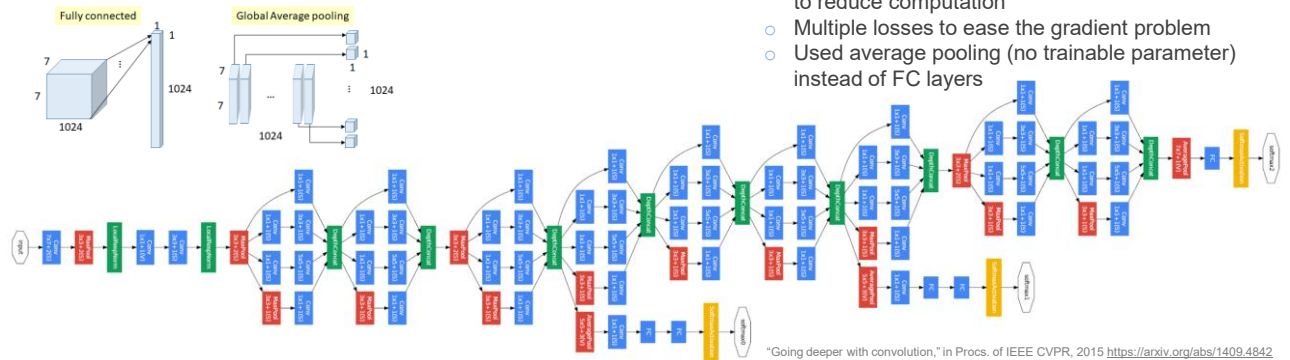


32

# A Inception v1 (ImageNet 2014, CVPR 2015)



- **GoogLeNet (Inception v1):** 22 layers
  - Winner of ImageNet 2014
  - Trained with Google's internal DL infrastructure, DistBelief (the distributed ML system)
- **Tricks:**
  - Inception module for parallel features
  - 1x1 convolution as a dimension reduction module to reduce computation
  - Multiple losses to ease the gradient problem
  - Used average pooling (no trainable parameter) instead of FC layers

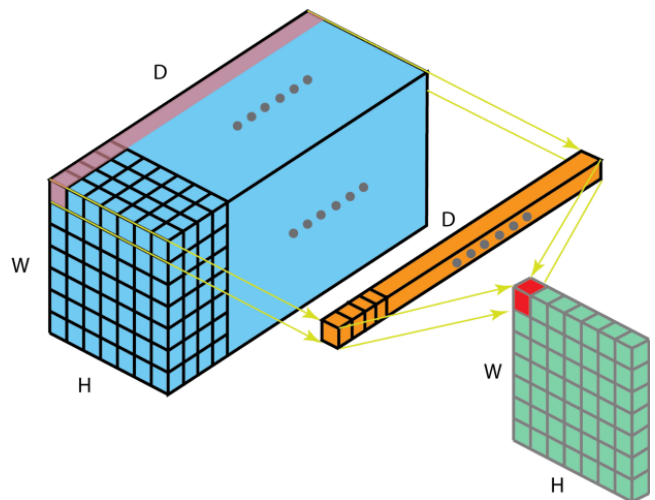


"Going deeper with convolution," in Procs. of IEEE CVPR, 2015 <https://arxiv.org/abs/1409.4842>

33

## 1x1 CONV2D

- **Advantages:**
  - Dimensionality reduction for efficient computations
  - Efficient low-dimensional embedding, or feature pooling
  - Applying nonlinearity again after convolution



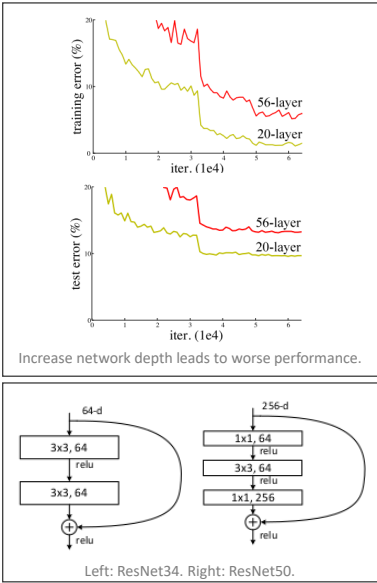
<https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

34

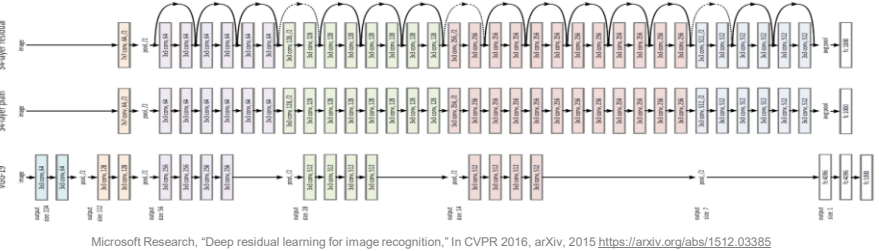


35

## Microsoft ResNet (ImageNet 2015, CVPR 2016)



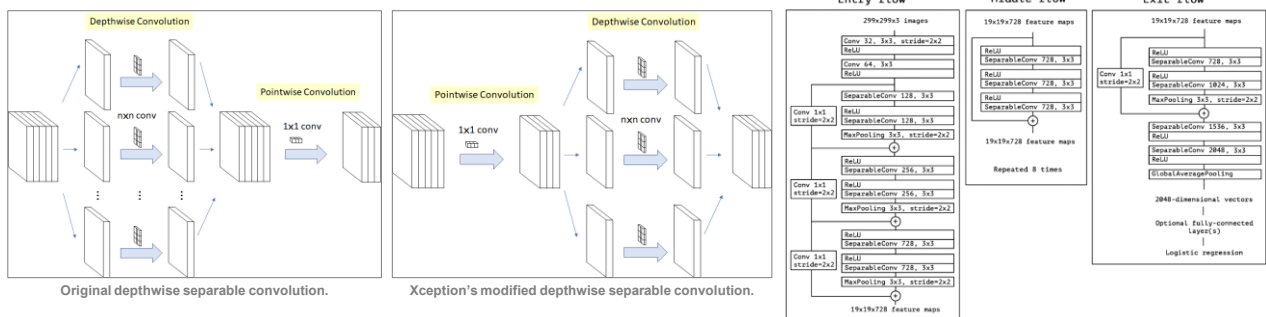
- **ResNet:** 152 layers
  - **ResNet-152** was the winner of ImageNet 2015.
  - Finished 90-epoch ImageNet-1k training with **ResNet-50** on a NVIDIA M40 GPU takes 14 days
  - ResNet-50 originally performed with a minibatch size of 256 images (using 8 Tesla P100 GPUs, training time is 29 hours).
  - In November 2018, researchers from Sony (Japan) trained ResNet-50 (50 layers) on ImageNet dataset with the new record of 224 seconds. This was done using the GPU cluster with 1,088 nodes (each node contains four NVIDIA Tesla V100 GPUs). [https://nnabla.org/paper/imagenet\\_in\\_224sec.pdf](https://nnabla.org/paper/imagenet_in_224sec.pdf)
- **Tricks:**
  - The core idea of ResNet is introducing a so-called "identity shortcut connection" that skips one or more layers.
    - Skip connections introduce shortcut paths for the gradient to flow through.
    - Ensure that the higher layer will perform at least as good as the lower layer, and not worse.



36

# Xception (CVPR 2017)

- **Xception** stands for **Ex**treme version of **Inception**.
  - Replace convolution in Inception with depthwise separable convolution to reduce computation.
  - **Xception** is comparable with Inception-v3 (i.e., 1st runner up in ImageNet 2015) but is much faster.

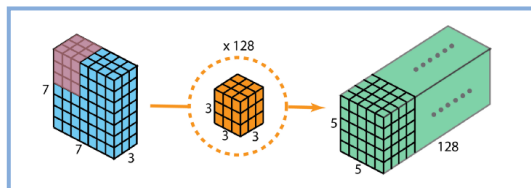


Francois Chollet, "Xception: Deep learning with depthwise separable convolutions," In Procs. of IEEE CVPR, 2017 <https://arxiv.org/abs/1610.02357>

37

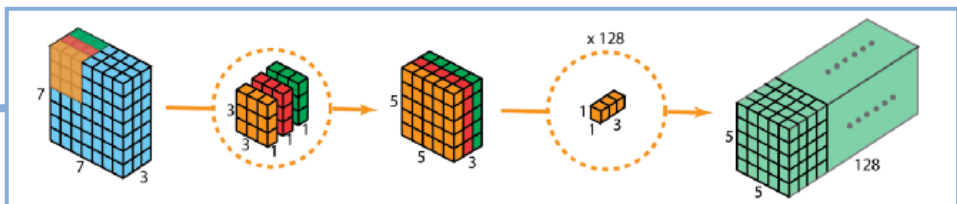
## Depthwise Separable Convolution

- **PRO:** Much less operations for depthwise separable convolutions compared to 2D convolutions
- **CON:** Reduce the number of parameters in the convolution, so that, for a small model, the model capacity may be decreased significantly and may become sub-optimal.



Standard 2D convolution to create output with 128 layer, using 128 filters.

The overall process of depthwise separable convolution.

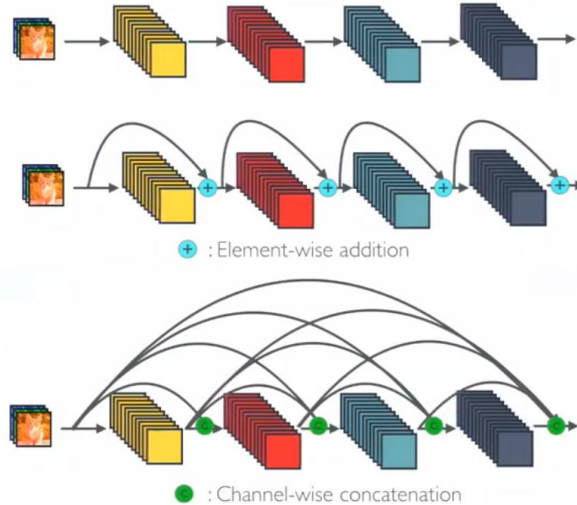


<https://towardsdatascience.com/a-comprehensive-introduction-to-different-types-of-convolutions-in-deep-learning-669281e58215>

38

# A DenseNet (CVPR 2017)

- **DenseNet** (best paper @CVPR2017) from Cornell university, Tsinghua university and Facebook AI



**Standard ConvNet concept:**  
In Standard ConvNet, input image goes through multiple convolution and obtain high-level features.

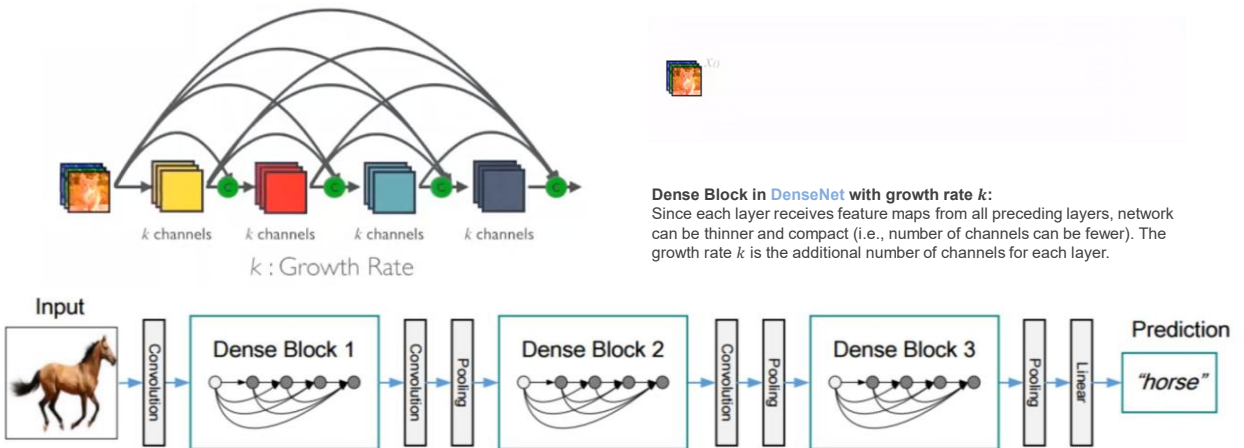
**ResNet concept:**  
Identity mapping is proposed to promote the gradient propagation. Element-wise addition is used. It can be viewed as algorithms with a state passed from one ResNet module to another one.

**One Dense Block in DenseNet:**  
Each layer obtains additional inputs from all preceding layers and passes on its own feature-maps to all subsequent layers. Concatenation is used. Each layer is receiving a "collective knowledge" from all preceding layers.

"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 <https://arxiv.org/abs/1608.06993> | Image credit: 25NOV2018 <https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803>

39

# A DenseNet (CVPR 2017)



**Dense Block in DenseNet with growth rate  $k$ :**  
Since each layer receives feature maps from all preceding layers, network can be thinner and compact (i.e., number of channels can be fewer). The growth rate  $k$  is the additional number of channels for each layer.

Figure 2. A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature map sizes via convolution and pooling.

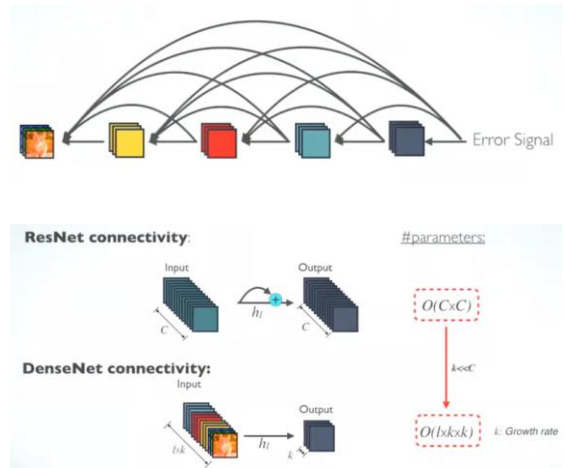
"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 <https://arxiv.org/abs/1608.06993> | Image credit: 25NOV2018 <https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803>

40

# A DenseNet (CVPR 2017)

## • PROs:

- *Strong gradient flow*: The error signal can be propagated to earlier layers more directly. This is a kind of implicit deep supervision as earlier layers can get direct supervision from the final classification layers.
- *Parameter and computational efficiency*: For each layer, number of parameters in ResNet is directly proportional to  $C \times C$  while number of parameters in **DenseNet** is directly proportional to  $l \times k \times k$ . Since  $k \ll C$ , **DenseNet** has much smaller size than ResNet.



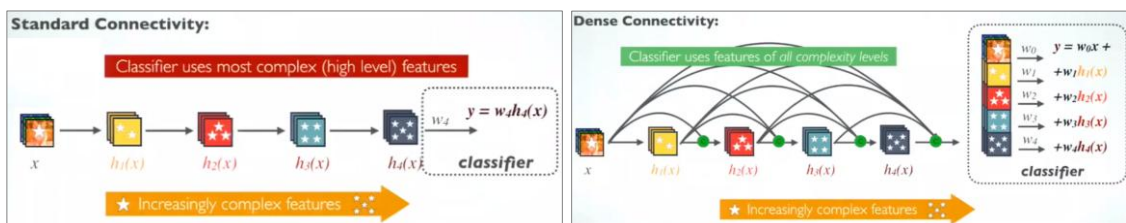
"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 <https://arxiv.org/abs/1608.06993> | Image credit: 25NOV2018 <https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803>

41

# A DenseNet (CVPR 2017)

## • PROs:

- *More diversified features*: Since each layer in **DenseNet** receive all preceding layers as input, more diversified features and tends to have richer patterns.
- *Maintain low complexity features*:
  - In a standard ConvNet, classifier uses most complex features.
  - In **DenseNet**, classifier uses features of all complexity levels. It tends to give more smooth decision boundaries. It also explains why **DenseNet** performs well when training data is insufficient.



"Densely connected convolutional networks," In Procs. of IEEE CVPR, 2017 <https://arxiv.org/abs/1608.06993> | Image credit: 25NOV2018 <https://towardsdatascience.com/review-densenet-image-classification-b6631a8ef803>

42



# Google NASNet (CVPR 2018)

- **NASNet** stands for **N**eural **A**rchitecture **S**earch **N**etwork: searching for the best network configuration using RL.
- **NASNet** achieves low(er) computation while maintaining high accuracy.

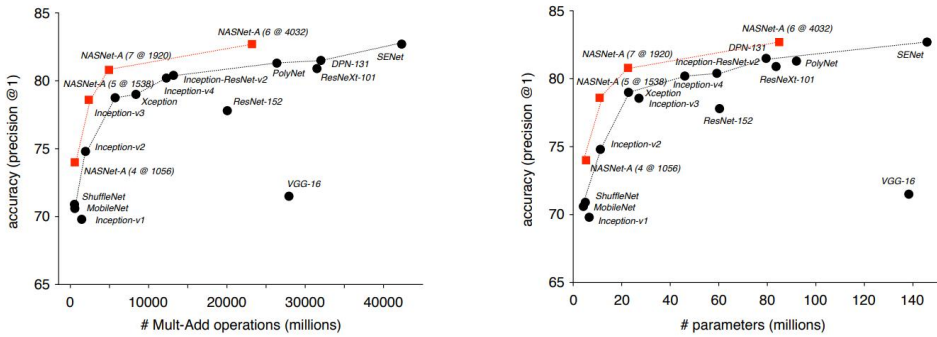


Figure 5. Accuracy versus computational demand (left) and number of parameters (right) across top performing published CNN architectures on ImageNet 2012 ILSVRC challenge prediction task. Computational demand is measured in the number of floating-point multiply-add operations to process a single image. Black circles indicate previously published results and red squares highlight our proposed models.

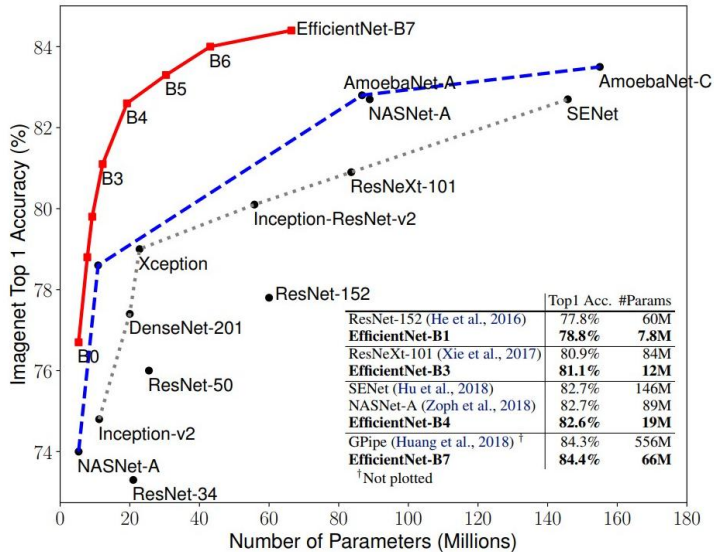
Google Research, "Learning Transferable Architectures for Scalable Image Recognition," In Proc. of IEEE/CVF CVPR, 2018  
<https://arxiv.org/abs/1707.07012>

43



# Google EfficientNet (ICML 2019)

- Google **EfficientNets** refer to a family of models which surpass state-of-the-art accuracy with up to 10x better efficiency (smaller and faster).



Google Brain, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," In Proc. of ICML, 2019 <https://arxiv.org/abs/1905.11946>

44

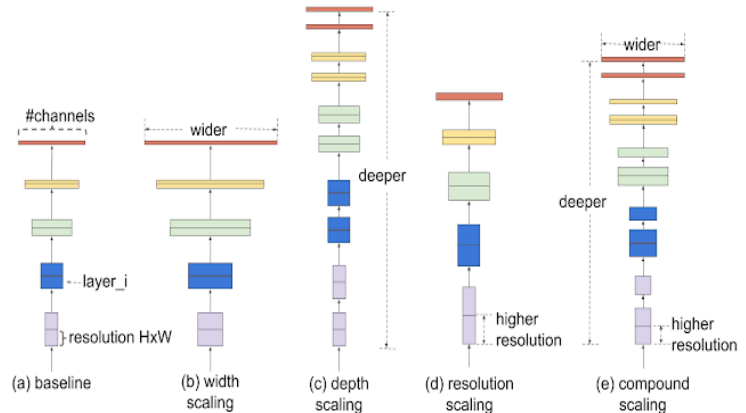
# A Google EfficientNet (ICML 2019)

- **Conventional model scaling:**

- The conventional practice is to arbitrarily increase the CNN depth or width, or to use larger input image resolution for training and evaluation.
- While these methods do improve accuracy, they usually require tedious manual tuning, and still often yield suboptimal performance.

- **Compound scaling method:**

- A novel model scaling method that uses a simple yet highly effective compound coefficient to scale up CNNs in a more structured manner.
- Unlike conventional approaches that arbitrarily scale network dimensions, such as width, depth and resolution, this method uniformly scales each dimension with a fixed set of scaling coefficients.
- This method consistently improves model accuracy and efficiency for scaling up existing models such as MobileNet (+1.4% imagenet accuracy), and ResNet (+0.7%), compared to conventional scaling methods.



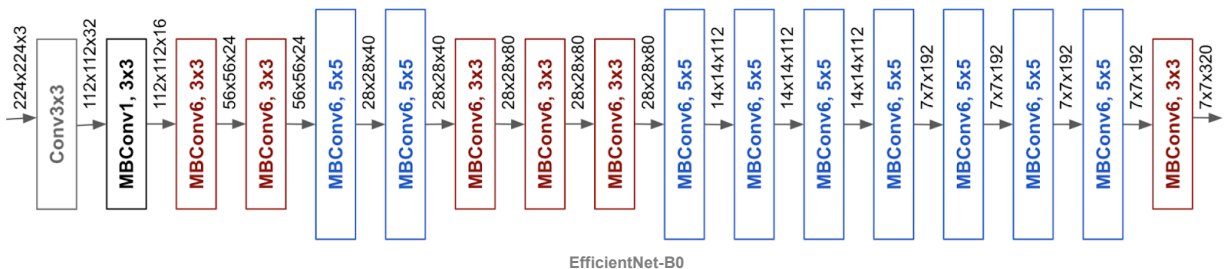
Google Brain, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," In Procs. of ICML, 2019 <https://arxiv.org/abs/1905.11946>

45

# A Google EfficientNet (ICML 2019)

- **Baseline network:**

- The effectiveness of model scaling also relies heavily on the baseline network.
- So, to further improve performance, a new baseline network is developed by performing a **Neural Architecture Search (NAS)** using the AutoML MNAS framework, which optimizes both accuracy and efficiency (FLOPS).
- The resulting architecture uses mobile inverted bottleneck convolution (MBConv), similar to MobileNetV2 and MnasNet, but is slightly larger due to an increased FLOP budget.
- A simple mobile-size baseline architecture developed by AutoML MNAS is called **EfficientNet-B0**, while **EfficientNet-B1** to **B7** are obtained by scaling up the baseline network.



Google Brain, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," In Procs. of ICML, 2019 <https://arxiv.org/abs/1905.11946>

46

# CNN image classifiers

## Classification

The following classification models are available, with or without pre-trained weights:

- AlexNet
- ConvNeXt
- DenseNet
- EfficientNet
- EfficientNetV2
- GoogLeNet
- Inception V3
- MaxViT
- MNASNet
- MobileNet V2
- MobileNet V3
- RegNet
- ResNet
- ResNeXt
- ShuffleNet V2
- SqueezeNet
- SwinTransformer
- VGG
- VisionTransformer
- Wide ResNet

```
from torchvision.io import decode_image
from torchvision.models import resnet50, ResNet50_Weights

img = decode_image("test/assets/encode_jpeg/grace_hopper_517x606.jpg")

# Step 1: Initialize model with the best available weights
weights = ResNet50_Weights.DEFAULT
model = resnet50(weights=weights)
model.eval()

# Step 2: Initialize the inference transforms
preprocess = weights.transforms()

# Step 3: Apply inference preprocessing transforms
batch = preprocess(img).unsqueeze(0)

# Step 4: Use the model and print the predicted category
prediction = model(batch).squeeze(0).softmax(0)
class_id = prediction.argmax().item()
score = prediction[class_id].item()
category_name = weights.meta["categories"][class_id]
print(f"{category_name}: {100 * score:.1f}%")
```

More available pre-trained weights at  
<https://docs.pytorch.org/vision/main/models.html#table-of-all-available-classification-weights>

<https://docs.pytorch.org/vision/main/models.html#classification>

47

# CNN image classifiers

## Table of all available classification weights

Accuracies are reported on ImageNet-1K using single crops:

| Weight                                           | Acc@1  | Acc@5  | Params | GFLOPS | Recip                |
|--------------------------------------------------|--------|--------|--------|--------|----------------------|
| AlexNet_Weights.IMAGENET1K_V1                    | 56.522 | 79.066 | 61.1M  | 0.71   | <a href="#">link</a> |
| ConvNeXt_Base_Weights.IMAGENET1K_V1              | 84.062 | 96.87  | 88.6M  | 15.36  | <a href="#">link</a> |
| ConvNeXt_Large_Weights.IMAGENET1K_V1             | 84.414 | 96.976 | 197.8M | 34.36  | <a href="#">link</a> |
| ConvNeXt_Small_Weights.IMAGENET1K_V1             | 83.616 | 96.65  | 50.2M  | 8.68   | <a href="#">link</a> |
| ConvNet_Tiny_Weights.IMAGENET1K_V1               | 82.52  | 96.146 | 28.6M  | 4.46   | <a href="#">link</a> |
| DenseNet121_Weights.IMAGENET1K_V1                | 74.434 | 91.972 | 8.0M   | 2.83   | <a href="#">link</a> |
| DenseNet161_Weights.IMAGENET1K_V1                | 77.138 | 93.56  | 28.7M  | 7.73   | <a href="#">link</a> |
| DenseNet169_Weights.IMAGENET1K_V1                | 75.6   | 92.806 | 14.1M  | 3.36   | <a href="#">link</a> |
| DenseNet201_Weights.IMAGENET1K_V1                | 76.896 | 93.37  | 20.0M  | 4.29   | <a href="#">link</a> |
| EfficientNet_B0_Weights.IMAGENET1K_V1            | 77.692 | 93.532 | 5.3M   | 0.39   | <a href="#">link</a> |
| EfficientNet_B1_Weights.IMAGENET1K_V1            | 78.642 | 94.186 | 7.8M   | 0.69   | <a href="#">link</a> |
| EfficientNet_B1_Weights.IMAGENET1K_V2            | 79.838 | 94.934 | 7.8M   | 0.69   | <a href="#">link</a> |
| EfficientNet_B2_Weights.IMAGENET1K_V1            | 80.608 | 95.31  | 9.1M   | 1.09   | <a href="#">link</a> |
| EfficientNet_B3_Weights.IMAGENET1K_V1            | 82.008 | 96.054 | 12.2M  | 1.83   | <a href="#">link</a> |
| EfficientNet_B4_Weights.IMAGENET1K_V1            | 83.384 | 96.594 | 19.3M  | 4.39   | <a href="#">link</a> |
| EfficientNet_B5_Weights.IMAGENET1K_V1            | 83.444 | 96.628 | 30.4M  | 10.27  | <a href="#">link</a> |
| EfficientNet_B6_Weights.IMAGENET1K_V1            | 84.008 | 96.916 | 43.0M  | 19.07  | <a href="#">link</a> |
| EfficientNet_B7_Weights.IMAGENET1K_V1            | 84.122 | 96.908 | 66.3M  | 37.75  | <a href="#">link</a> |
| EfficientNet_V2_L_Weights.IMAGENET1K_V1          | 85.808 | 97.788 | 118.5M | 56.08  | <a href="#">link</a> |
| EfficientNet_V2_H_Weights.IMAGENET1K_V1          | 85.112 | 97.156 | 54.1M  | 24.58  | <a href="#">link</a> |
| EfficientNet_V2_S_Weights.IMAGENET1K_V1          | 84.228 | 96.878 | 21.5M  | 8.37   | <a href="#">link</a> |
| GoogLeNet_Weights.IMAGENET1K_V1                  | 69.778 | 89.53  | 6.6M   | 1.5    | <a href="#">link</a> |
| Inception_V3_Weights.IMAGENET1K_V1               | 77.294 | 93.45  | 27.2M  | 5.71   | <a href="#">link</a> |
| MNASNet0_5_Weights.IMAGENET1K_V1                 | 67.734 | 87.49  | 2.2M   | 0.1    | <a href="#">link</a> |
| MNASNet0_75_Weights.IMAGENET1K_V1                | 71.18  | 90.496 | 3.2M   | 0.21   | <a href="#">link</a> |
| MNASNet1_0_Weights.IMAGENET1K_V1                 | 73.456 | 91.51  | 4.4M   | 0.31   | <a href="#">link</a> |
| MNASNet1_3_Weights.IMAGENET1K_V1                 | 76.506 | 93.522 | 6.3M   | 0.53   | <a href="#">link</a> |
| MaxViT_T_Weights.IMAGENET1K_V1                   | 83.7   | 96.722 | 30.9M  | 5.56   | <a href="#">link</a> |
| MobileNet_V2_Weights.IMAGENET1K_V1               | 71.878 | 90.286 | 3.5M   | 0.3    | <a href="#">link</a> |
| MobileNet_V2_Weights.IMAGENET1K_V2               | 72.154 | 90.822 | 3.5M   | 0.3    | <a href="#">link</a> |
| MobileNet_V3_Large_Weights.IMAGENET1K_V1         | 74.042 | 91.34  | 5.5M   | 0.22   | <a href="#">link</a> |
| MobileNet_V3_Large_Weights.IMAGENET1K_V2         | 75.274 | 92.566 | 5.5M   | 0.22   | <a href="#">link</a> |
| MobileNet_V3_Small_Weights.IMAGENET1K_V1         | 67.668 | 87.402 | 2.5M   | 0.06   | <a href="#">link</a> |
| MobileNet_V3_Small_Weights.IMAGENET1K_V2         | 80.058 | 94.944 | 54.3M  | 15.94  | <a href="#">link</a> |
| RegNet_X_160F_Weights.IMAGENET1K_V1              | 82.716 | 96.196 | 54.3M  | 15.94  | <a href="#">link</a> |
| RegNet_X_160F_Weights.IMAGENET1K_V2              | 77.04  | 93.44  | 9.2M   | 1.6    | <a href="#">link</a> |
| RegNet_X_1_60F_Weights.IMAGENET1K_V1             | 79.668 | 94.922 | 9.2M   | 1.6    | <a href="#">link</a> |
| RegNet_X_1_60F_Weights.IMAGENET1K_V2             | 80.622 | 95.248 | 107.8M | 31.74  | <a href="#">link</a> |
| RegNet_X_320F_Weights.IMAGENET1K_V1              | 83.014 | 96.288 | 107.8M | 31.74  | <a href="#">link</a> |
| RegNet_X_320F_Weights.IMAGENET1K_V2              | 78.364 | 93.992 | 15.3M  | 3.18   | <a href="#">link</a> |
| RegNet_Y_1280F_Weights.IMAGENET1K_V1             | 81.196 | 95.43  | 15.3M  | 3.18   | <a href="#">link</a> |
| RegNet_Y_1280F_Weights.IMAGENET1K_V2             | 74.864 | 92.322 | 5.5M   | 0.41   | <a href="#">link</a> |
| RegNet_Y_400HF_Weights.IMAGENET1K_V1             | 72.834 | 90.95  | 5.5M   | 0.41   | <a href="#">link</a> |
| RegNet_Y_400HF_Weights.IMAGENET1K_V2             | 74.864 | 92.322 | 5.5M   | 0.41   | <a href="#">link</a> |
| RegNet_Y_800HF_Weights.IMAGENET1K_V1             | 75.212 | 92.348 | 7.3M   | 0.8    | <a href="#">link</a> |
| RegNet_Y_800HF_Weights.IMAGENET1K_V2             | 77.522 | 93.826 | 7.3M   | 0.8    | <a href="#">link</a> |
| RegNet_Y_80F_Weights.IMAGENET1K_V1               | 79.344 | 94.686 | 39.6M  | 8      | <a href="#">link</a> |
| RegNet_Y_80F_Weights.IMAGENET1K_V2               | 81.682 | 95.678 | 39.6M  | 8      | <a href="#">link</a> |
| RegNet_Y_1280F_Weights.IMAGENET1K_SWAG_E2E_V1    | 88.228 | 98.682 | 644.8M | 374.57 | <a href="#">link</a> |
| RegNet_Y_1280F_Weights.IMAGENET1K_SWAG_LINEAR_V1 | 86.068 | 97.844 | 644.8M | 127.52 | <a href="#">link</a> |
| RegNet_Y_160F_Weights.IMAGENET1K_V1              | 80.424 | 95.24  | 83.6M  | 15.91  | <a href="#">link</a> |
| RegNet_Y_160F_Weights.IMAGENET1K_V2              | 82.886 | 96.328 | 83.6M  | 15.91  | <a href="#">link</a> |
| RegNet_Y_160F_Weights.IMAGENET1K_SWAG_E2E_V1     | 86.012 | 98.054 | 83.6M  | 46.73  | <a href="#">link</a> |
| RegNet_Y_160F_Weights.IMAGENET1K_SWAG_LINEAR_V1  | 83.976 | 97.244 | 83.6M  | 15.91  | <a href="#">link</a> |
| RegNet_Y_1_60F_Weights.IMAGENET1K_V1             | 77.95  | 93.966 | 11.2M  | 1.61   | <a href="#">link</a> |

<https://docs.pytorch.org/vision/main/models.html#table-of-all-available-classification-weights>

48

# K CNN image classifiers

- Losses
- Data loading
- Built-in small datasets
- Keras Applications
- Mixed precision
- Utilities
- KerasTuner
- Code examples
- Why choose Keras?
- Community & governance
- Contributing to Keras
- KerasTuner

| Model             | Size (MB) | Top-1 Accuracy | Top-5 Accuracy | Parameters | Depth | Time (ms) per inference step (CPU) | Time (ms) per inference step (GPU) |
|-------------------|-----------|----------------|----------------|------------|-------|------------------------------------|------------------------------------|
| Xception          | 88        | 79.0%          | 94.5%          | 22.9M      | 81    | 109.4                              | 8.1                                |
| VGG16             | 528       | 71.3%          | 90.1%          | 138.4M     | 16    | 69.5                               | 4.2                                |
| VGG19             | 549       | 71.3%          | 90.0%          | 143.7M     | 19    | 84.8                               | 4.4                                |
| ResNet50          | 98        | 74.9%          | 92.1%          | 25.6M      | 107   | 58.2                               | 4.6                                |
| ResNet50V2        | 98        | 76.0%          | 93.0%          | 25.6M      | 103   | 45.6                               | 4.4                                |
| ResNet101         | 171       | 76.4%          | 92.8%          | 44.7M      | 209   | 89.6                               | 5.2                                |
| ResNet101V2       | 171       | 77.2%          | 93.8%          | 44.7M      | 205   | 72.7                               | 5.4                                |
| ResNet152         | 232       | 76.6%          | 93.1%          | 60.4M      | 311   | 127.4                              | 6.5                                |
| ResNet152V2       | 232       | 78.0%          | 94.2%          | 60.4M      | 307   | 107.5                              | 6.6                                |
| InceptionV3       | 92        | 77.9%          | 93.7%          | 23.9M      | 189   | 42.2                               | 6.9                                |
| InceptionResNetV2 | 215       | 80.3%          | 95.3%          | 55.9M      | 449   | 130.2                              | 10.0                               |
| MobileNet         | 16        | 70.4%          | 89.5%          | 4.3M       | 55    | 22.6                               | 3.4                                |
| MobileNetV2       | 14        | 71.3%          | 90.1%          | 3.5M       | 105   | 25.9                               | 3.8                                |
| DenseNet121       | 33        | 75.0%          | 92.3%          | 8.1M       | 242   | 77.1                               | 5.4                                |
| DenseNet169       | 57        | 76.2%          | 93.2%          | 14.3M      | 338   | 96.4                               | 6.3                                |
| DenseNet201       | 80        | 77.3%          | 93.6%          | 20.2M      | 402   | 127.2                              | 6.7                                |
| NASNetMobile      | 23        | 74.4%          | 91.9%          | 5.3M       | 389   | 27.0                               | 6.7                                |
| NASNetLarge       | 343       | 82.5%          | 96.0%          | 88.9M      | 533   | 344.5                              | 20.0                               |
| EfficientNetB0    | 29        | 77.1%          | 93.3%          | 5.3M       | 132   | 46.0                               | 4.9                                |
| EfficientNetB1    | 31        | 79.1%          | 94.4%          | 7.9M       | 186   | 60.2                               | 5.6                                |

|                  |        |       |       |        |     |        |      |
|------------------|--------|-------|-------|--------|-----|--------|------|
| EfficientNetB2   | 36     | 80.1% | 94.9% | 9.2M   | 186 | 80.8   | 6.5  |
| EfficientNetB3   | 48     | 81.6% | 95.7% | 12.3M  | 210 | 140.0  | 8.8  |
| EfficientNetB4   | 75     | 82.9% | 96.4% | 19.5M  | 258 | 308.3  | 15.1 |
| EfficientNetB5   | 118    | 83.6% | 96.7% | 30.6M  | 312 | 579.2  | 25.3 |
| EfficientNetB6   | 166    | 84.0% | 96.8% | 43.3M  | 360 | 958.1  | 40.4 |
| EfficientNetB7   | 256    | 84.3% | 97.0% | 66.7M  | 438 | 1578.9 | 61.6 |
| EfficientNetV2B0 | 29     | 78.7% | 94.3% | 7.2M   | -   | -      | -    |
| EfficientNetV2B1 | 34     | 79.8% | 95.0% | 8.2M   | -   | -      | -    |
| EfficientNetV2B2 | 42     | 80.5% | 95.1% | 10.2M  | -   | -      | -    |
| EfficientNetV2B3 | 59     | 82.0% | 95.8% | 14.5M  | -   | -      | -    |
| EfficientNetV2S  | 88     | 83.9% | 96.7% | 21.6M  | -   | -      | -    |
| EfficientNetV2M  | 220    | 85.3% | 97.4% | 54.4M  | -   | -      | -    |
| EfficientNetV2L  | 479    | 85.7% | 97.5% | 119.0M | -   | -      | -    |
| ConvNeXTTiny     | 109.42 | 81.3% | -     | 28.6M  | -   | -      | -    |
| ConvNeXTSmall    | 192.29 | 82.3% | -     | 50.2M  | -   | -      | -    |
| ConvNeXTBase     | 338.58 | 85.3% | -     | 88.5M  | -   | -      | -    |
| ConvNeXTLarge    | 755.07 | 86.3% | -     | 197.7M | -   | -      | -    |
| ConvNeXTLarge    | 1310   | 86.7% | -     | 350.1M | -   | -      | -    |

The top-1 and top-5 accuracy refers to the model's performance on the ImageNet validation dataset.

Depth refers to the topological depth of the network. This includes activation layers, batch normalization layers etc.

Time per inference step is the average of 30 batches and 10 repetitions.

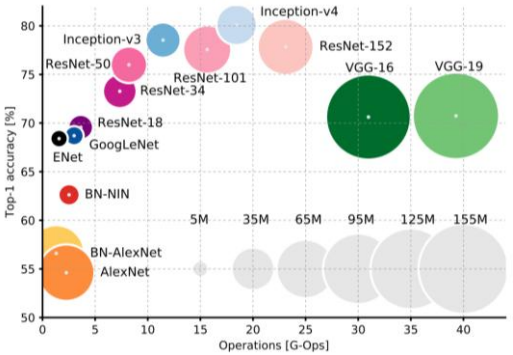
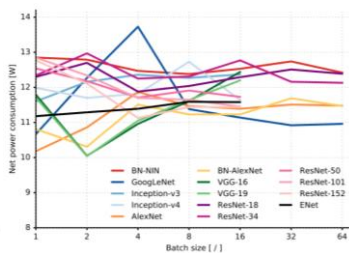
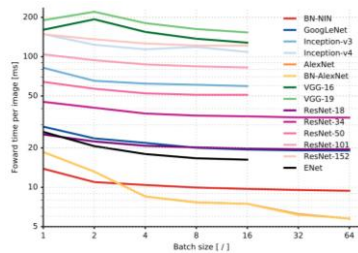
- CPU: AMD EPYC Processor (with IBPB) (92 core)
- RAM: 1.7T
- GPU: Tesla A100
- Batch size: 32

Depth counts the number of layers with parameters.

<https://keras.io/applications/>

49

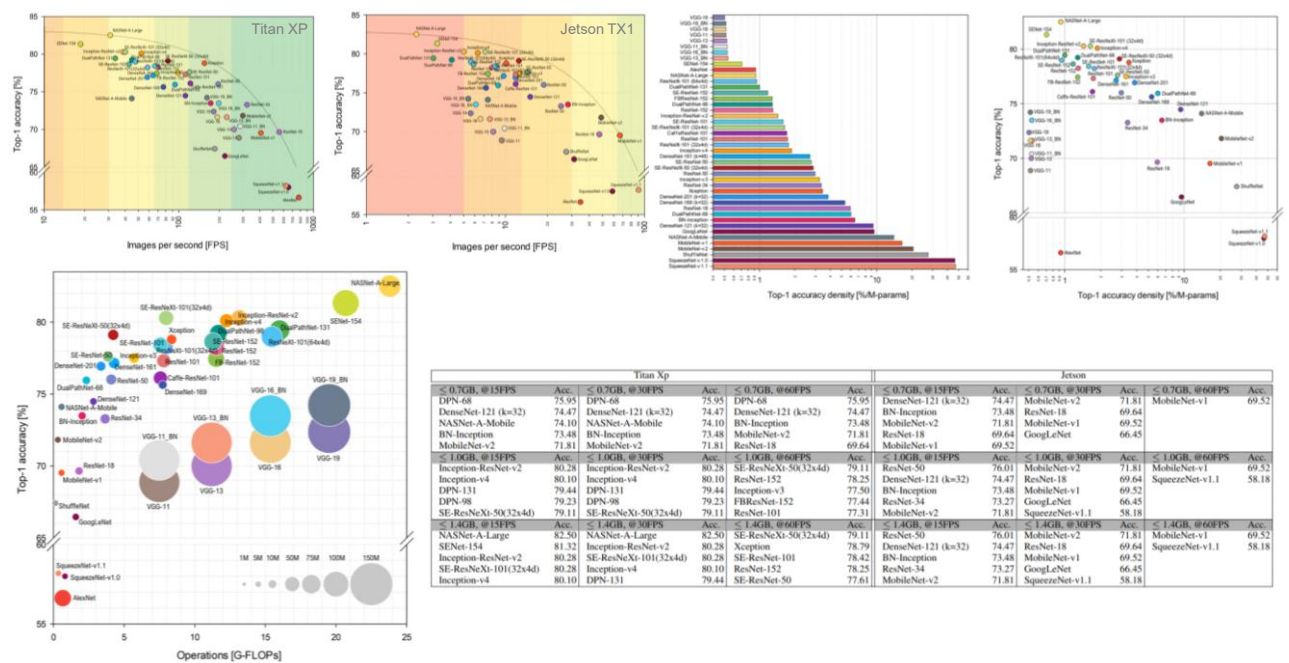
## Which CNN to choose?



- **Accuracy:** how precise the model
- **FLOPs:** how many operations to perform
- **Model size:** how big is the model
- **RAM usage:** how much RAM the model consumes
- **Inference time:** how fast the model predicts an input
- **Power consumption:** how much power is used to run the model
- Note that small models do not automatically convey a small amount of RAM memory and FLOPs.

Canziani, Culurciello, and Paszke, "An Analysis of Deep Neural Network Models for Practical Applications," In arXiv, 2017: <https://arxiv.org/abs/1605.07678>

50



“Benchmark Analysis of Representative Deep Neural Network Architectures,” In IEEE Access, 2018 <https://arxiv.org/abs/1810.00736>

51

# EX2: The pre-trained image classifier

- Load and use a SOTA image classifier that is already pre-trained on ImageNet dataset



Top 10 predictions:

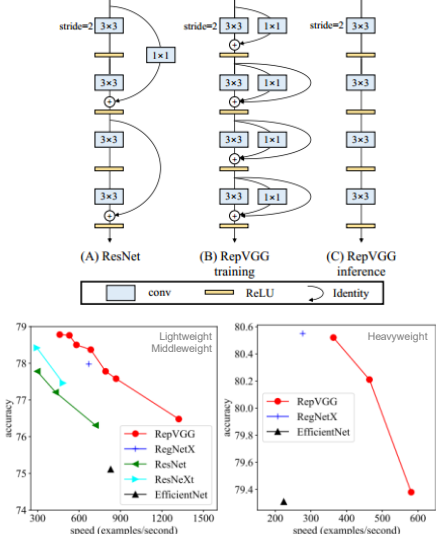
```
1: ['n02112018', 'Pomeranian'] (Probability: 0.5500)
2: ['n02112350', 'keeshond'] (Probability: 0.0317)
3: ['n02086079', 'Pekinese'] (Probability: 0.0063)
4: ['n02112137', 'chow'] (Probability: 0.0052)
5: ['n02085782', 'Japanese spaniel'] (Probability: 0.0035)
6: ['n04041544', 'radio'] (Probability: 0.0034)
7: ['n02086910', 'papillon'] (Probability: 0.0030)
8: ['n02111889', 'Samoyed'] (Probability: 0.0027)
9: ['n04409515', 'tennis_ball'] (Probability: 0.0020)
10: ['n02097474', 'Tibetan_terrier'] (Probability: 0.0017)
```

52



53

# A RepVGG (CVPR 2021)



Thitirat TheSensei  
5 กุมภาพันธ์ 2021

RepVGG: Make VGG great again

Intro: State-of-the-art VGG

พูดถึงชื่อ VGG ผลงานจากทีมวิจัย Oxford Visual Geometry Group เชื่อว่าคนสาย image/vision ส่วนใหญ่ต้องร้อง อ้อ เพราะ VGG network คือ State-of-the-art (SOTA) CNN (Convolutional Neural Network) ตัวแรก ๆ ของยุค modern deep learning (DL) ที่โด่งดังจากการคว้าตำแหน่งชนะเลิศอันดับ 1 ในการแข่งขัน ImageNet 2014 (classification task)

จากปี 2014 วาร्षกลับมาถึงปัจจุบันปี 2021 หากมองในแง่ CNN รุ่นใหม่ที่เกิดขึ้นย่อมมีมากมาย ไม่น่าเชื่อว่า VGG ที่ยังคงสามารถยืนหยัดเป็นหนึ่งใน backbone ตัวเด็ดของยุคสำหรับสายงาน image/vision อยู่ได้ ... โดยจุดหนึ่งที่ VGG แตกต่างจากคนอื่น (ในมุมมอง) อย่างเห็นได้ชัด คือ ความเป็น network ที่เรียบง่าย ๆ ที่มีแค่ a stack of 3x3 convolution and ReLU เรียงกันตรงไปตรงมา ไม่มี skip connection ไม่มีทางแยก ไม่มี branch ไม่มีการแตกกิ่งก้านสาขาอะไรใน network ทั้งสิ้น

เชื่อว่าถ้าใช้ DL framework ยุคปัจจุบันอย่าง TensorFlow หรือ PyTorch ... แม้แต่มือใหม่เพิ่งหัดเขียน CNN ก็ใช้ Sequential API เขียนโปรแกรมเขียนแบบ VGG architecture (from scratch) เสร็จได้ภายในไม่กี่นาที (ไม่นับเวลา train นะ) ... หรือถ้าอยากไปทางสายคล่องกว่าขึ้นอีก ก็เรียก pre-trained VGG ขึ้นมาใช้ได้เลย ใน DL framework มีเตรียมมาให้พร้อมใช้แล้ว

What's wrong with multi-branch designs?

ถ้าเปรียบเทียบ CNN backbones สุดคลาสสิกอย่าง VGG vs. ResNet ค่าแนะนำที่มักบอกต่อ ๆ กัน คือ ถ้าค่าความเร็วและประสิทธิภาพเท่าที่ศึกษา ควรเลือก ResNet ที่มี skip connection เนื่องจาก "a multi-branch architecture makes the model an implicit ensemble of numerous shallower models" ... ส่วน VGG นั้น ในแง่ความแม่นยำก็อาจเป็นรอง แต่ข้อดีที่เป็นโครงสร้างที่เรียบง่าย เข้าใจง่าย (เนื่องจาก low-level features ค่อนข้างง่าย ไม่ high-level features ไม่มีขั้วขึ้น ซึ่งก็ขึ้นกับนิยามของงานบางอย่างโดยเฉพาะ เช่น การทำ style transfer)

<https://www.facebook.com/thitirat.thelecturer/posts/2824859631067610>

"RepVGG: Making VGG-style ConvNets Great Again," In Procs. of CVPR, 2021 <https://arxiv.org/abs/2101.03697>

54

# DeepMind NFNet (ICLR 2021)

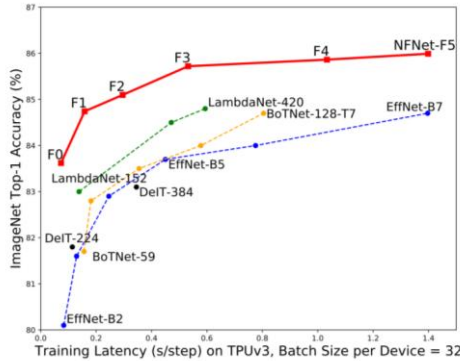


Figure 1. ImageNet Validation Accuracy vs Training Latency. All numbers are single-model, single crop. Our NFNet-F1 model achieves comparable accuracy to an EffNet-B7 while being 8.7× faster to train. Our NFNet-F5 model has similar training latency to EffNet-B7, but achieves a state-of-the-art 86.0% top-1 accuracy on ImageNet. We further improve on this using Sharpness Aware Minimization (Foret et al., 2021) to achieve 86.5% top-1 accuracy.

Thitirat TheSensei  
15 กุมภาพันธ์ 2021 · 🌐

#ICLR2021 DeepMind NFNet เด็ดกว่า เร็วกว่า เก่งกว่าแชมป์เก่า Google EfficientNet 1d ด้วยโมเดลของ network without normalization (no batch norm)

#สรุปแล้วก็มียาวๆ #เนื้อหาลาก2ป่ไปต่อกว่าๆ

Google EfficientNet (ICML2019) vs. (Google) DeepMind NFNet (ICLR2021)

กลายเป็นศึกชิงบัลลังก์กันเองภายใน Google ละแล้ว เปรียบเทียบ SOTA (state-of-the-art) CNN (Convolutional Neural Network) image classifier คนล่าสุดบน ImageNet dataset อย่าง Google EfficientNet ที่เราทั่วไป ก็เพิ่งขึ้นบัลลังก์ก็ครองมงกุฎได้ไม่นานเท่าไรเองนะ แลวันนี่โดนรุ่นน้องส่งประกาดโดย DeepMind (ซึ่งก็เป็นทีมหนึ่งของ Google เช่นกัน) ชื่อ NFNet มาตะโกนว่าที่ ขออภัยคุณเดวิดที่เก่งทุกและบัลลังก์ก็ไม่ใช่ของคุณแล้ว

ผลงาน NFNet นี้ถ้าจะจะได้ตีพิมพ์ในงาน conference ต้องระดับโลกอย่าง ICLR2021 ซึ่งที่จัดงานจริงวันที่ 4-8 พค 2021 โบน แต่ช่วงนี้คือ preprint (เผยแพร่) ICLR2021 เสร็จออกมาแล้ว ลากตามไถ่คนได้ข่าวก็บอกแล้ว ... อย่าง preprint NFNet นี้ถ้าจะครองบัลลังก์จริงๆ 3 ประการ เรายกจุดความสนใจของคนในวงการ image/vision ได้เป็นอย่างดี น่าจะโดนใจคนต้องอ่านหาอ่านหน่อย

... ความมีประสบการณ์ เด็ดกว่า DeepMind ก็การันตีฝีมือด้าน AI ระดับนี้วางอันดับต้น ๆ ของโลกแล้ว 🤖 ซิอีส์ดังแต่สมัย AlphaGo เป็นสนาม เสนออะไรมาก็มีแต่มีง ๆ โลกคือสิ่งทั้งนั้น (ข้อมูลผลงานบางส่วนในโพสต์เก่า ๆ ที่แท็ก #DeepMind 1d)

... ความมีประสบการณ์ที่ต่อเนื่อง คือ DeepMind นี้ก็ด้านมาก ที่ออกมาด้านนี้แบบหลักเทคโนโลยีระดับด้านของวงการ DL อย่าง BatchNorm ซะเต็ม ๆ หนา ๆ ขนาน ๆ (แบบมีดเคียวหอบเป็นทีที่เห็นคือคือสิ่งนั้นเลย) ... คืออย่างนี้ว่าไม่พอ ยังดูส่าหมีตะลุยไปวงการ DL ต่อสู้กับ move on รู้จักไปต่ออย่าง สิ่งนี้ที่เคยมองในอดีตว่าน่าจะมี และถ้าไปสู้สิ่งใหม่ ๆ ที่ศึกษาอีกต่างหาก 🤖

... ความมีประสบการณ์ที่สาม แชมป์มีจวบจนอย่าง Google EfficientNet ถูกแทนที่ซะไรอันไรได้ยังไม่ ... แต่แบบสับสนแบบคำยังเคยมา NFNet นี้ทั้งเด็ดกว่า เร็วกว่า (หรือนได้เร็วกว่า EfficientNet ถึง 8.7x) แถมแม่นยำกว่า EfficientNet และสามารถทำสถิติใหม่บน ImageNet dataset ได้ด้วยนะเธอ (ภาพประกอบภาพแรกได้โพส) ... ไม่พอยังบอกว่าตัวเคมเป็นแบบมาเลย NFNet 1d ง่าย transfer learning/fine tuning คือดีกว่าพวกอื่น network

➤ <https://www.facebook.com/thitirat.thelecturer/posts/2831757527044487>

Google DeepMind, "High-Performance Large-Scale Image Recognition Without Normalization," In arXiv, 2021 <https://arxiv.org/abs/2102.06171>  
Google DeepMind, "Characterizing signal propagation to close the performance gap in unnormalized ResNets," In Procs. of ICLR, 2021 <https://arxiv.org/abs/2101.08692>

55



56



# Foundation model

Jump start our CNNs with the big foundation models and their pre-trained weights

57

## AI THE MODERN HISTORY

- **2006:** Hinton's unsupervised layer-wise **pretraining** technique reignited the DL term in the research community



- **2012:** ImageNet evolution—**AlexNet** 🐉

- **2014:** Birth of **GRU**, **VAE**, and **GAN**

- **2015:** Birth of **diffusion model**

- **2017:** The **DeepFake** viral
- **2017:** Birth of the groundbreaking **Transformer**



- **2018:** NLP's ImageNet moment—**BERT**
- **2018:** ACM Turing Award to Deep NN



58

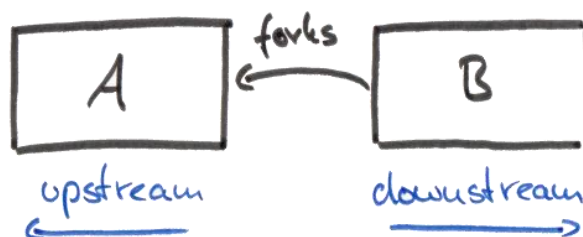
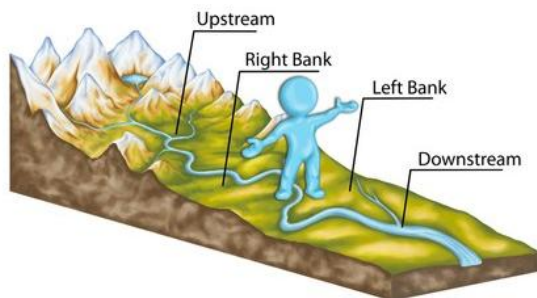
# A Downstream / Upstream tasks

- **Downstream task:**

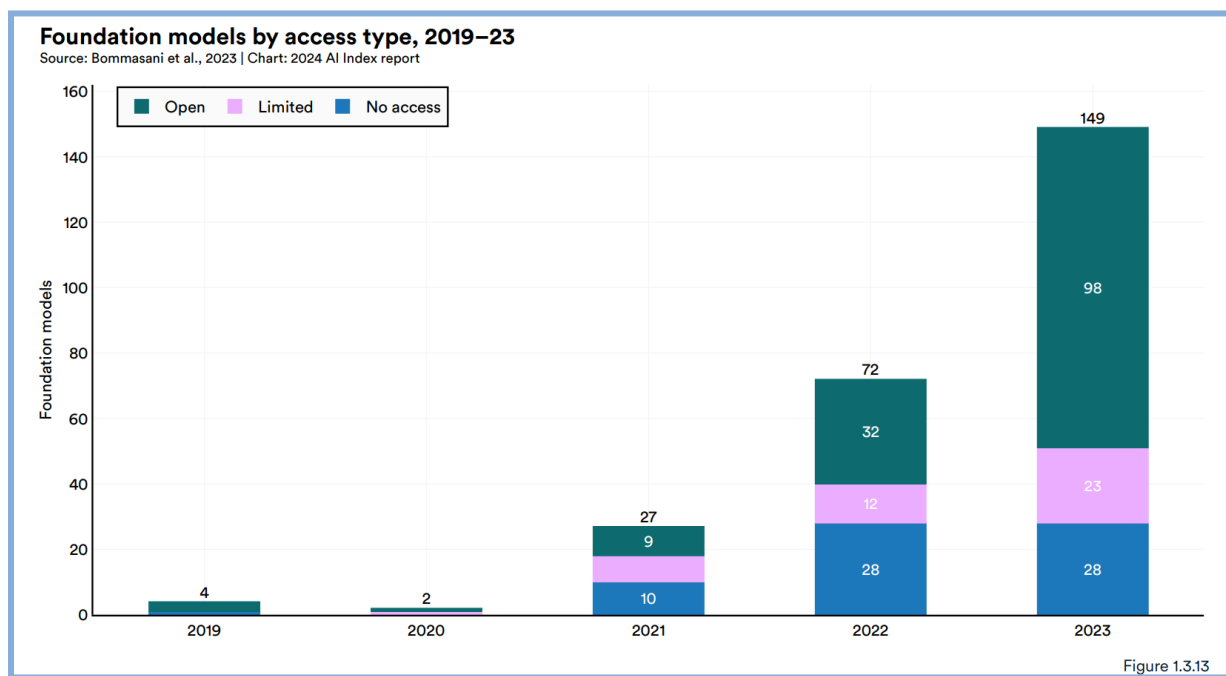
- This is what the deep learning field calls those supervised-learning tasks that utilize a pre-trained model or component. <http://jalammar.github.io/illustrated-bert/>

- **Upstream task:**

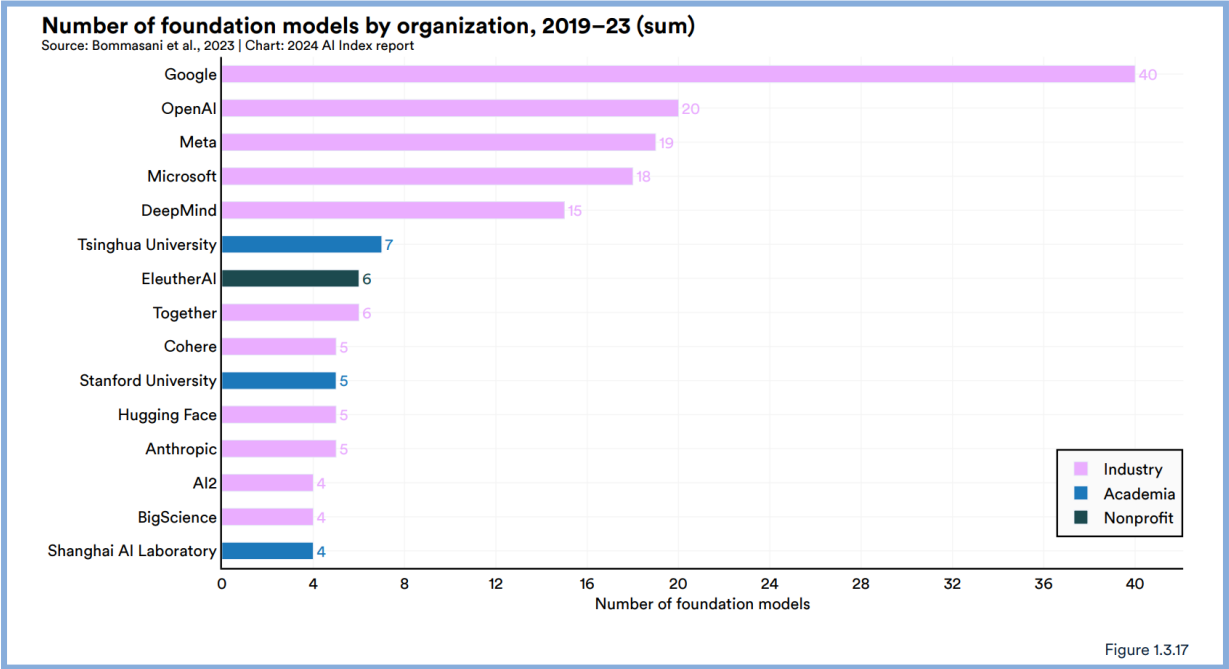
- In the context of open source projects, **A** is the upstream project since it can very well live without project B. But project B (the fork, the downstream project) wouldn't even exist without project A (the original project). <https://reflectoring.io/upstream-downstream/>



59

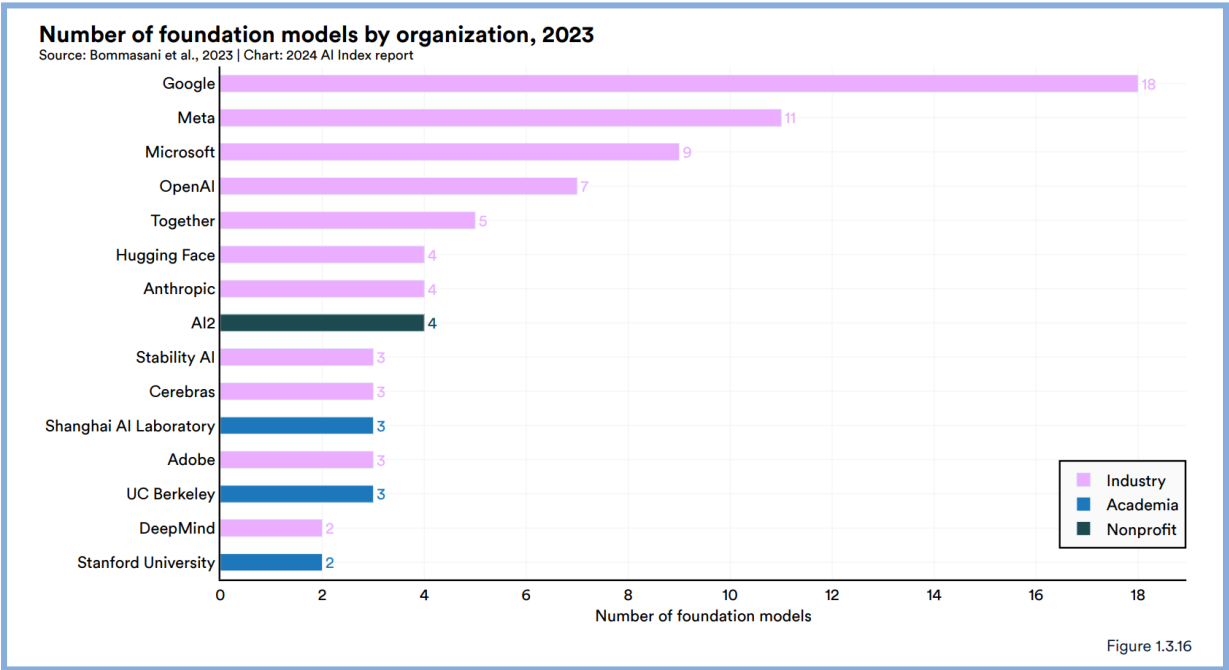
HAI AI Index Report 2024 (April 2024), <https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI-AI-Index-Report-2024.pdf>

60



HAI AI Index Report 2024 (April 2024), [https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI\\_AI-Index-Report-2024.pdf](https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI_AI-Index-Report-2024.pdf)

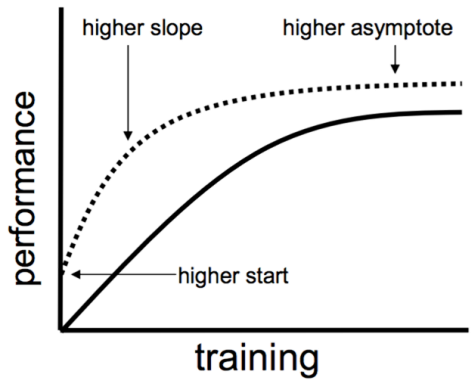
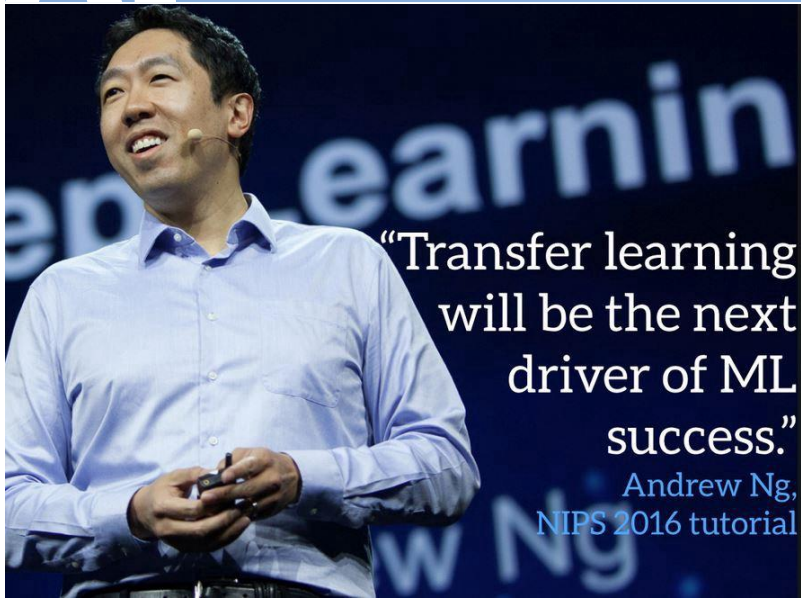
61



HAI AI Index Report 2024 (April 2024), [https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI\\_AI-Index-Report-2024.pdf](https://aiindex.stanford.edu/wp-content/uploads/2024/04/HAI_AI-Index-Report-2024.pdf)

62

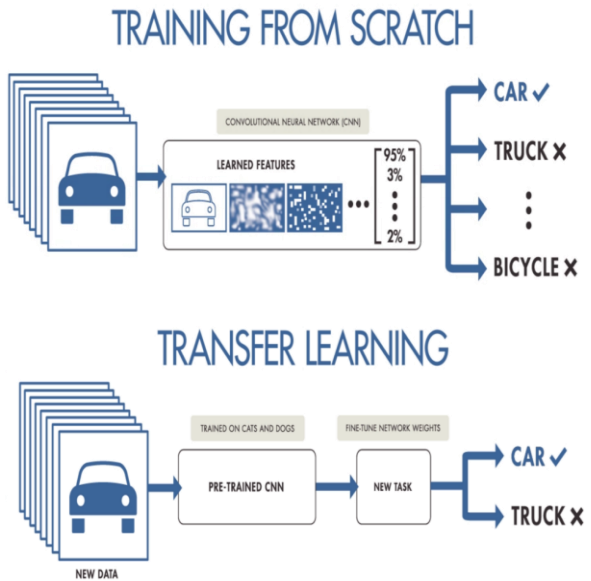
# AI Transfer Learning



..... with transfer  
— without transfer

63

# AI Transfer Learning



64

# Finetuning

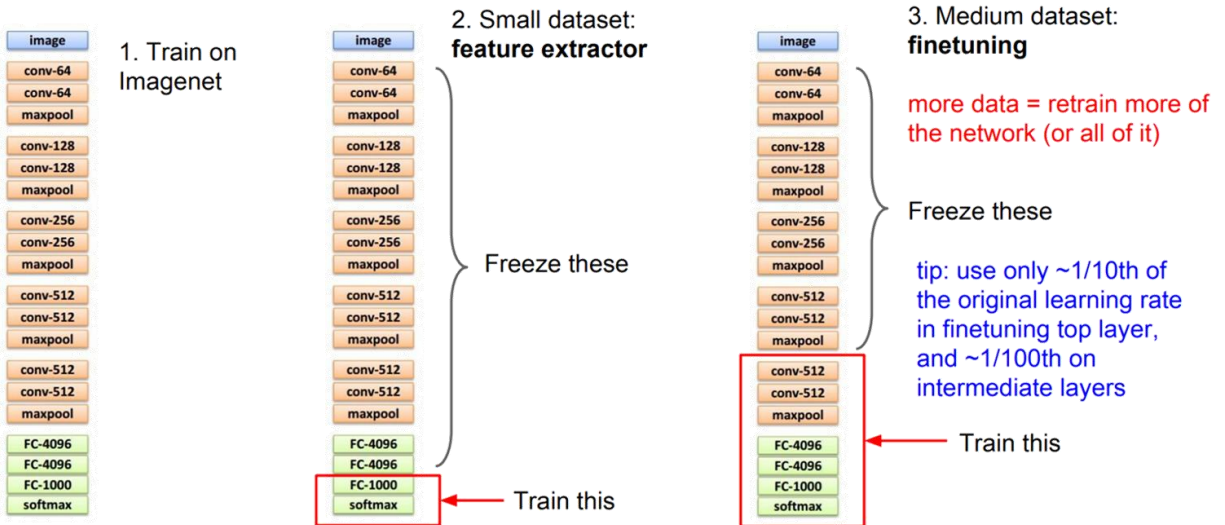


Image credit: [http://cs231n.stanford.edu/slides/2016/winter1516\\_lecture11.pdf](http://cs231n.stanford.edu/slides/2016/winter1516_lecture11.pdf)

65

## Visual Feature hierarchy

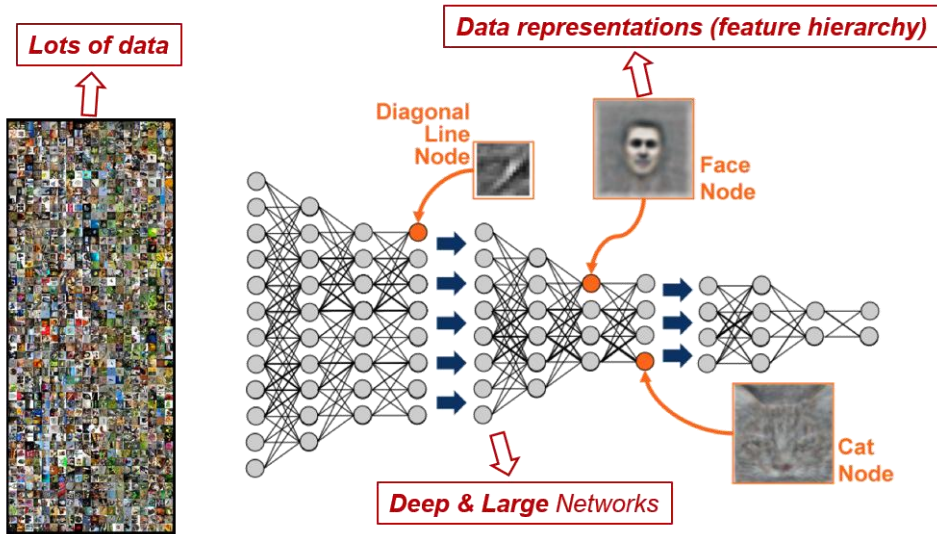
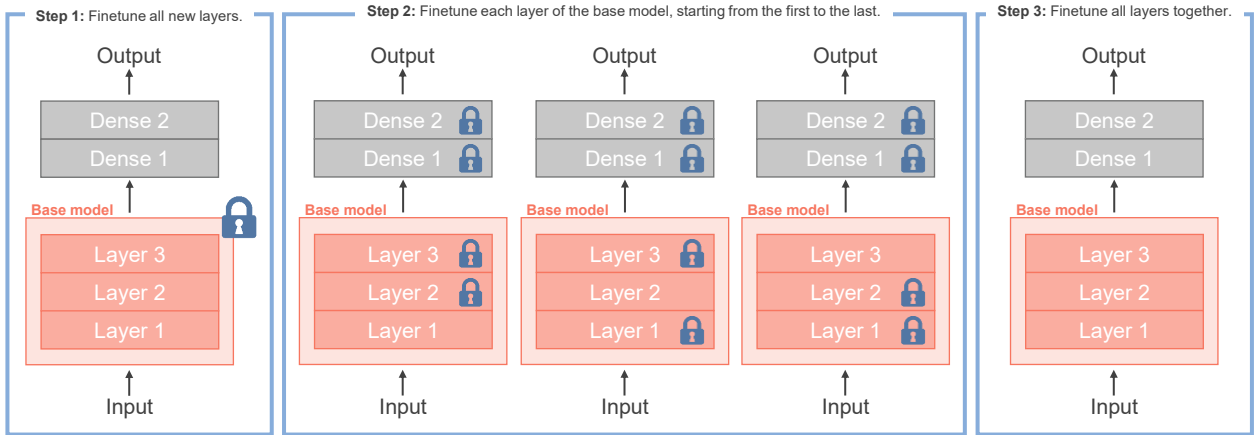


Image credit: <http://brain.kaist.ac.kr/research.html>

66

# Chain-Thaw Finetuning (2017)

- A transfer learning approach that sequentially unfreezes and finetunes a single layer at a time.
- Increase accuracy on the target task at the expense of extra computational power needed for the finetuning.

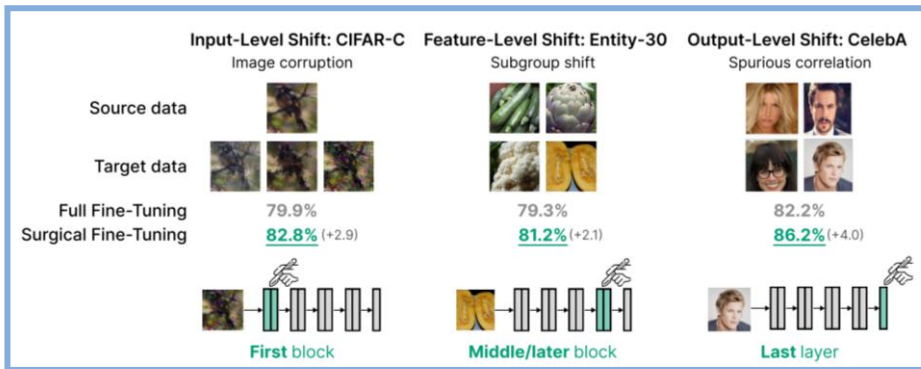


Felbo et al., "Using millions of emoji occurrences to learn any-domain representations for detecting sentiment, emotion and sarcasm," In Procs. of EMNLP 2017 <https://arxiv.org/abs/1708.00524>

67

# Surgical Finetuning (ICLR 2023)

- A common approach to transfer learning under distribution shift is to finetune the last few layers of a pre-trained model. This is to fit the new data while also preserving the information obtained during the pre-training phase.
- **Surgical finetuning** shows that selectively finetuning a subset of layers matches or outperforms commonly used finetuning approaches.

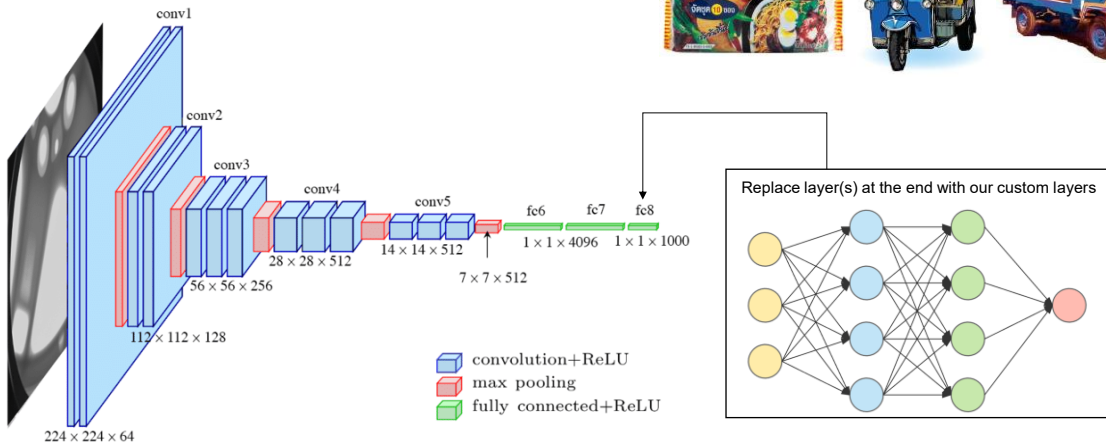


Stanford university, "Surgical Fine-Tuning Improves Adaptation to Distribution Shifts," arXiv 2022, ICLR 2023 <https://arxiv.org/abs/2210.11466>

68

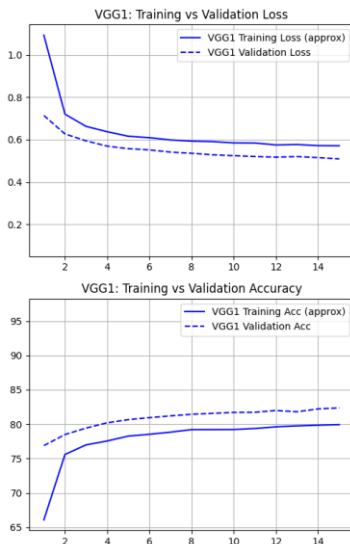
## EX3: Downstream image classifier

- Different domain (e.g., ImageNet vs. CIFAR-10) but same task (image classification)



69

## EX3: Downstream image classifier

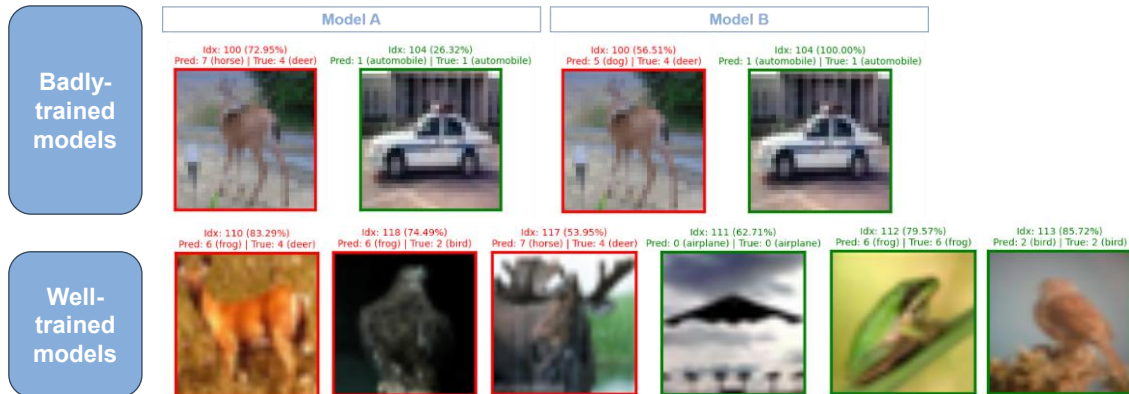


- Data preprocessing:
  - Normalization**: use ImageNet stats, not CIFAR-10 stats.
  - Image resize**:
    - The size of the input image must match the backbone's requirements (i.e.,  $224 \times 224$  for VGG16).
    - Despite no error, don't directly feed  $32 \times 32$  images to the VGG16 backbone.
  - Training transformation**: define manually to include our custom augmentation.
  - Test transformation**: can reuse the backbone's transformation.
- Why does validation performance exceed training performance?
  - Evaluation timing
  - Data augmentation
  - Training regularization

70

## EX3: Downstream image classifier

- **Model calibration:** Accuracy and confidence are related, but they measure different things.
  - Cross-entropy training encourages the model to assign higher probability to the correct class, but it does not guarantee that the predicted probabilities reflect the true likelihood of being correct.
  - In a well-trained model, confidence and correctness are often associated.



71

## Be Careful ...

- How big is our dataset? How similar our dataset compared to the original dataset?

|           | Small                           | Large                                                                                                           |
|-----------|---------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Similar   | Train only the (last) FC layer. | Freeze the earlier layers (simple features) and train the rest of layers.                                       |
| Different | Train only the FC layers.       | Train the model from scratch and reuse the network architecture (using the trained weights as a started point). |

- Should we re-train the existing layer(s) or add the new layer(s)?
- What architecture to be used in the newly added layers?
- One or multiple transfer learnings?

Thitirat TheSensei  
27 มกราคม เวลา 13:39 น. · 🌐

#MetaAI ประยุกต์ใช้ AI มาสร้าง animation จากภาพวาดตัวการ์ตูนในจินตนาการของเด็ก ๆ

เมื่อช่วงกลางเดือนธันวาคม 2021 เห็นคุณ Mark Zuckerberg แห่ง Meta แพร่เว็บ <https://sketch.metademolab.com/canvas> นี่มา ... ลองเข้าไปกดเล่นกับตัว sample figure แล้วก็บารักดั้นะ ดูตัวการ์ตูนลายเส้นเด่นกับกระดิกกระดิก

พอได้อ่านรายละเอียดการพัฒนาใน blog แล้ว ก็เจอประเด็นน่าสนใจหลายประเด็นที่ไม่ใช่การเสนอ state-of-the-art (SOTA) ตัวใหม่ แต่เป็นการชี้ให้เห็นถึงจุดอ่อนของพวก pretrained vision models (deep learning: DL) ใน task ปกติที่คนชอบทำกันอย่าง object detection และ segmentation ที่สุดท้ายก็ต้องวนกลับมาใช้ classical techniques ที่ใช้ human knowledge นี่สะท้อนไปช่วยอุดรูโหว่ของปัญหาไปพลาง ๆ (จนกว่าจะมี training data มากและหลากหลายครอบคลุมพอ) ... ว่าแล้วเลยมาเขียนสรุปเอาไว้สักหน่อย

\*\*\*\*\*

📌 Why kid's drawings are difficult?

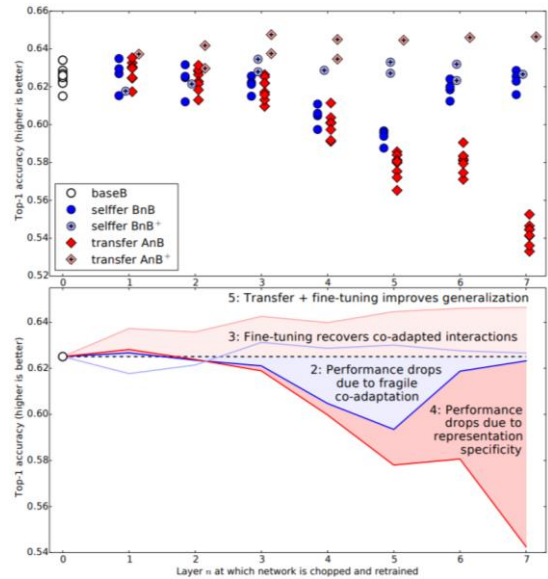
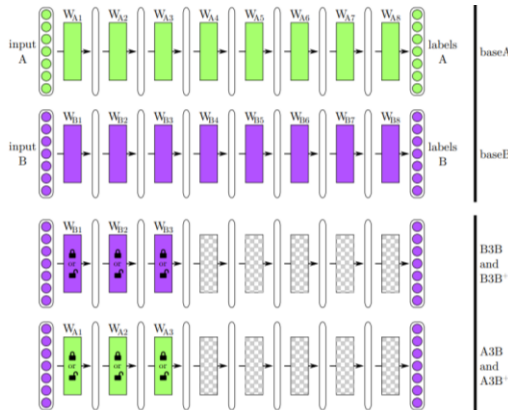
ในปัจจุบันด้วยอาณานิสงส์ของ SOTA DL model หลายตัวที่ถูก pretrain มาให้อย่างดีแล้ว การจะทำ object detection หรือ segmentation บนภาพ คน สัตว์ หรือ figure อะไรสักอย่างเลยดูจะไม่ใช่เรื่องยาก

<https://www.facebook.com/thitirat.thelecturer/posts/3096122700607967>

72

# A Co-adaptation issue

- When neurons on neighboring layers co-adapt during training in such a way that cannot be rediscovered when one layer is frozen.



Yosinski et al., "How transferable are features in deep neural networks?," NIPS 2014 <https://arxiv.org/abs/1411.1792>

73

# A Catastrophic Forgetting / Negative Transfer

- Catastrophic Forgetting:**
  - Achieving artificial general intelligence requires that intelligent agents must demonstrate a capacity for **continual learning**—the ability to learn in a sequential fashion, accumulating the knowledge learnt in previous tasks, and using it to help future learning.
  - Continual learning** is challenging for neural networks as neural networks tend to abruptly lose previously learnt knowledge (of task A) as information relevant to the current task (task B) is incorporated. This phenomenon is called **Catastrophic Forgetting**.
  - Continue reading: "Overcoming catastrophic forgetting in neural networks," arXiv 2DEC2016, <https://arxiv.org/abs/1612.00796>.
- Negative Transfer:**
  - When labelled data is scarce for a specific target task, transfer learning often offers an effective solution by utilizing data from a related source task.
  - However, when transferring knowledge from a less related source, it may inversely hurt the target performance, a phenomenon known as **Negative Transfer**.
  - Continue reading: "Characterizing and avoiding negative transfer," CVPR2019, <https://ieeexplore.ieee.org/document/8953565>.

74

# Assess transferability of a pre-trained model

- **Linear probing:**
  - Attach a linear classifier to the pre-trained model's feature extractor.
  - Then, train the model (while freezing the feature extractor) on a task-specific dataset and evaluate the performance.
  - This method is simple and straightforward but is limited to linear classifiers and may not capture complex relationships.
- **Finetuning:**
  - Unfreeze some or all of the pre-trained model's layers and train to update their weights using task-specific data.
- Many more ideas can be found in the latest publications that propose training a new foundation model.



75



## Demystify CNN

Make CNNs more transparent and  
visually explainable

76

# Why debugging CNN?

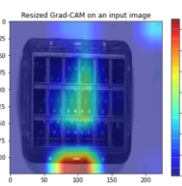
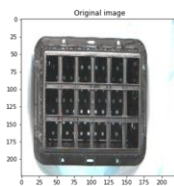
- How can we trust the decisions of a model if we cannot properly validate how it arrived there?
  - Where the network is looking in the input image?
  - Which series of neurons activated in the forward pass during inference/prediction?
  - How the network arrived at its final output?
  - ...



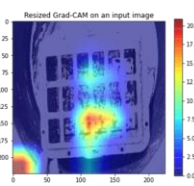
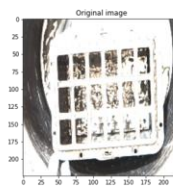
- **For example:**
  - We may accidentally create a CNN model that classifies human genders by looking at the background color instead of looking at a human's face.
  - However, this CNN may yield 100% accuracy on both train and test sets.



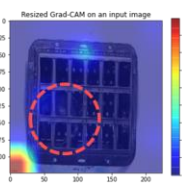
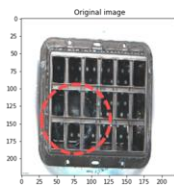
77



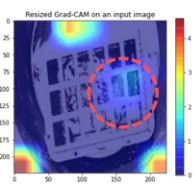
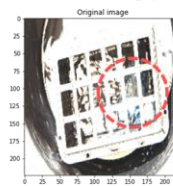
Empty basket Indoor



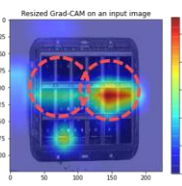
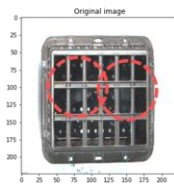
Empty basket Outdoor



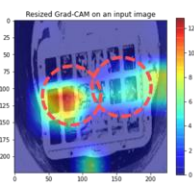
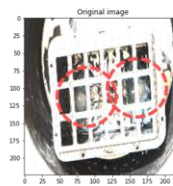
One crab Indoor



One crab Outdoor



Two crabs Indoor



Two crabs Outdoor



W. Siripattanadilok and T. Siriborvornratanakul, "Recognition of Partially Occluded Soft-shell Mud Crabs Using Deep Learning-based Image Classification and Object Detection," Aquaculture International, 2024

78

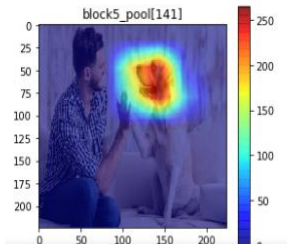
# A Blackbox AI vs. eXplainable AI

- **Explainability** means that it's possible to "explain" how a model made its decision.
  - **BUT** the explanation is not guaranteed to make sense to humans.
  - **AND** the explanation is not constrained to follow any known rules of the natural world.
  - **AND** the explanation is not guaranteed to be fair or free from biases.
  - For example:
    - A model may "explain" a boat classification by highlighting the water.
    - A model may "explain" a severely ill classification by highlighting a label within a medical image that indicates that the image was taken while the patient was lying down.
- **Interpretability** means that a model has been designed from the beginning to produce a human-understandable relationship between the inputs and the outputs.
  - For example:
    - Logistic regression is an interpretable model, in which the design of the model results in weights that show which inputs contribute more or less to the final predictions.
    - Rule-based methods are also interpretable.
- **Explainability** is not the same as **interpretability!!!**

79

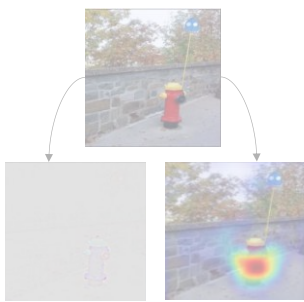
# A Visualizing what CNN learns

## INTERMEDIATE ACTIVATIONS



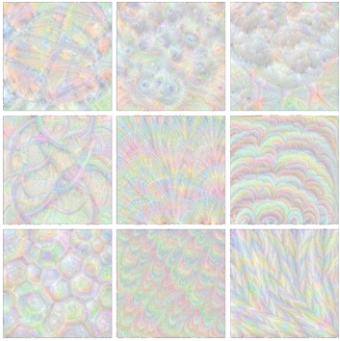
- Useful for understanding how successive convolutional layers transform their input
- Useful for getting a first idea of the meaning of individual convolutional filters

## WHERE EACH CLASS LOOKS



- Useful for understanding which parts of an image were identified as belonging to a given class, allowing us to localize objects in images

## CNN FILTERS



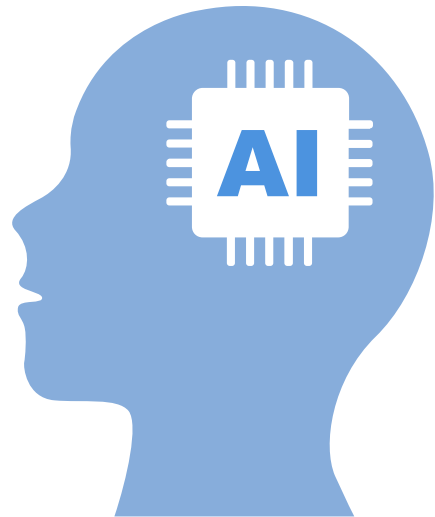
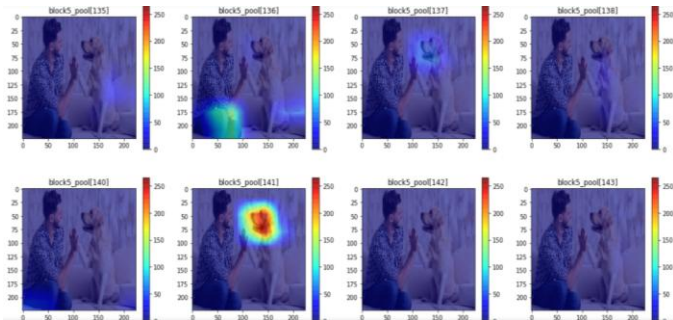
- Useful for understanding precisely what visual pattern or concept each filter in a CNN is receptive to

80

# QUESTION

## INTERMEDIATE ACTIVATIONS

For a given channel/filter, which parts of the input produce the highest activation?



81

# Visualizing what CNN learns

## INTERMEDIATE ACTIVATIONS

- Useful for understanding how successive convolutional layers transform their input
- Useful for getting a first idea of the meaning of individual convolutional filters

## WHERE EACH CLASS LOOKS

- Useful for understanding which parts of an image were identified as belonging to a given class, allowing us to localize objects in images

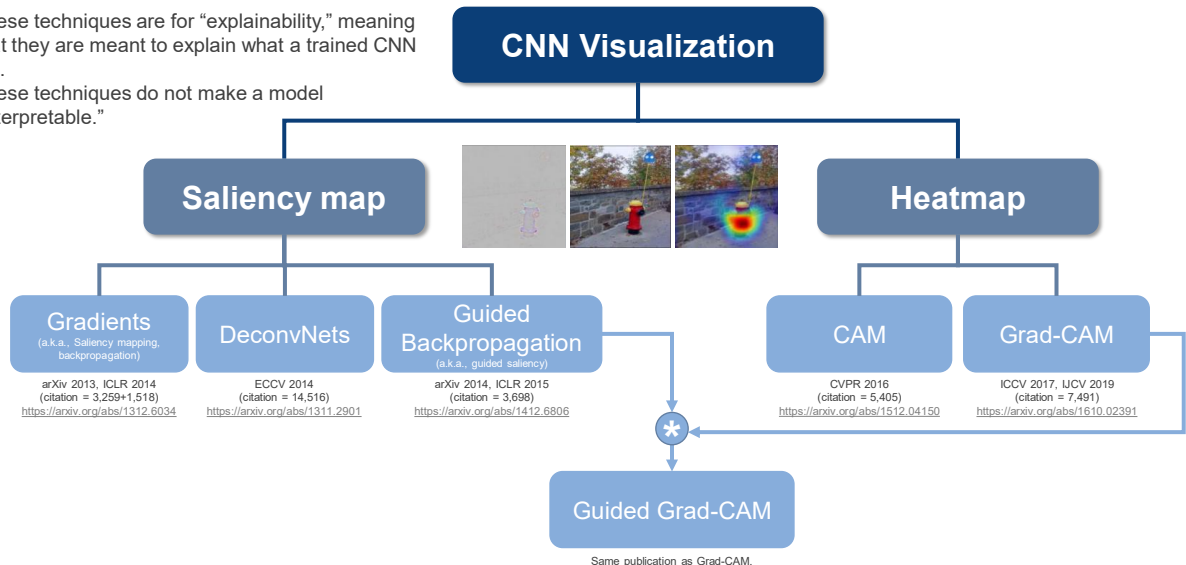
## CNN FILTERS

- Useful for understanding precisely what visual pattern or concept each filter in a CNN is receptive to

82

# Visualizing where each class looks

- These techniques are for “explainability,” meaning that they are meant to explain what a trained CNN did.
- These techniques do not make a model “interpretable.”



All numbers of citations from Google Scholar on February 7, 2022

83

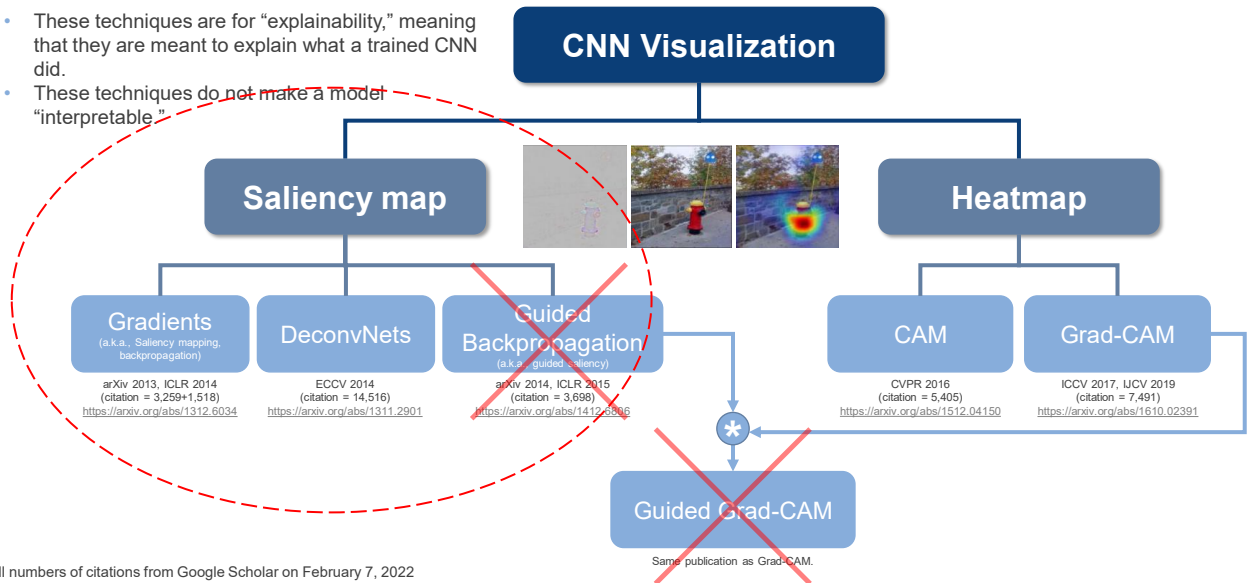
# Visualizing where each class looks

- **Sanity checks** refer to methods of determining whether a particular technique provides insight into a model or not.
- Two works from NeurIPS2018 <https://arxiv.org/pdf/1810.03292> and ICML2018 <https://arxiv.org/abs/1805.07039> suggest that:
  - Several popular saliency map approaches do not in fact provide insight into what a model is doing. Models with randomized weights and models trained on data with randomized labels will still produce convincing-looking “explanations” for certain techniques.
- **Gradients and Grad-CAM pass the sanity checks, while Guided Backpropagation and Guided Grad-CAM fail.**
  - To use saliency map (or heatmap) explanations, stick to Gradients or vanilla Grad-CAM, as they appear to rely on the trained model’s weights and the relationship between training examples and their labels.
  - It is best to avoid Guided Backpropagation and Guided Grad-CAM, as they may merely be functioning as edge detectors rather than reliable explanations.

84

# Visualizing where each class looks

- These techniques are for “explainability,” meaning that they are meant to explain what a trained CNN did.
- These techniques do not make a model “interpretable.”



All numbers of citations from Google Scholar on February 7, 2022

85

# Saliency Map

- A **saliency map** is any visualization of an image in which the most salient/pixels are highlighted.
  - There are traditional computer vision saliency detection algorithms (e.g., OpenCV saliency API).
  - However, the focus of this class will be on saliency maps created from trained CNNs.
- The purpose of the **saliency map** approach is to query an already-trained classification CNN about the spatial support of a particular class in a particular image.
  - For example, figure out where the cat is in the cat photo without any explicit location labels

|                                     | Target class: King Snake (56) | Target class: Mastiff (243) | Target class: Spider (72) |
|-------------------------------------|-------------------------------|-----------------------------|---------------------------|
| Original Image                      |                               |                             |                           |
| Colored Vanilla Backpropagation     |                               |                             |                           |
| Colored Guided Backpropagation (GB) |                               |                             |                           |

86

# A Saliency Map

- **Gradients** (a.k.a., saliency mapping, backpropagation) (2013), **DeconvNets** (2014), and **Guided Backpropagation** (a.k.a., guided saliency) (2014) are three closely related methods for producing saliency maps from an already-trained CNN.
- **Gradients**, **DeconvNets**, and **Guided Backpropagation** have been used for weakly supervised object localization (because the approximate position of the object is highlighted) and for gaining insight into a network's misclassifications.
  - The three methods share the common idea of computing the gradient of the network's prediction with respect to the input, holding the weights fixed. This determines which input elements (i.e., which pixels in case of an input image) need to be changed the least to affect the prediction the most.
  - The sole difference between the three methods is how they backpropagate through the linear rectifier (ReLU).
  - However, only **Gradients** method actually computes the gradient, whereas the others modify the backpropagation step to do something slightly different.

87

# A Saliency Map

- **Gradients** method was introduced in the paper entitled "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps." (Oxford VGG @arXiv2013) <https://arxiv.org/abs/1312.6034>
  1. Compute  $\frac{\partial S_c}{\partial I}$ , when  $S_c$  is the unnormalized class score (i.e., prediction score before softmax) regarding the output class  $C$ , and  $I$  is an input image.
  2. For an input image of  $N$  channels, the partial differentiation in step 1 returns  $N$  intermediate maps, namely,  $map_0, map_1, map_2, \dots, map_{N-1}$
  3. Suppose that  $SaliencyMap(i, j)$  refers to the pixel  $(i, j)$  of the saliency map and  $map_k(i, j)$  refers to the pixel  $(i, j)$  of the intermediate map corresponding to channel  $k$ , then
    - If  $N = 1$ ,  $SaliencyMap(i, j) = |map_0(i, j)|$
    - If  $N > 1$ ,  $SaliencyMap(i, j) = \max(|map_0(i, j)|, |map_1(i, j)|, \dots, |map_{N-1}(i, j)|)$

88

# A Saliency Map

- **Gradients** vs. **DeconvNets** vs. **Guided Backpropagation** :
  - **DeconvNets** method is the same as **Gradients** method except that **DeconvNets** only backpropagates positive error signals (i.e., set the negative gradients to zero).
  - **Guided Backpropagation** method combines **Gradients** and **DeconvNets** when handling the ReLU nonlinearity:
    - Like **DeconvNets**, **Guided Backpropagation** only backpropagates positive error signals.
    - Like **Gradients**, **Guided Backpropagation** also restricts to only positive inputs.
    - Thus, the gradient is “guided” by both the input and the error signal.

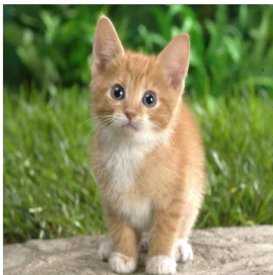
For more details about similarities and differences of the three methods, please continue reading in (6OCT2019)  
<https://glassboxmedicine.com/2019/10/06/cnn-heat-maps-gradients-vs-deconvnets-vs-guided-backpropagation/>

89

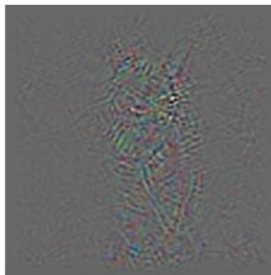
# A Saliency Map

- The gradient of the loss for the “cat” class with respect to the input pixels (i.e., **Gradients** method) gives a noisy output whereas **Guided Backpropagation** method gives much cleaner result.

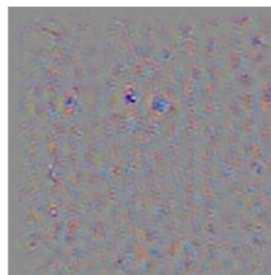
Input image



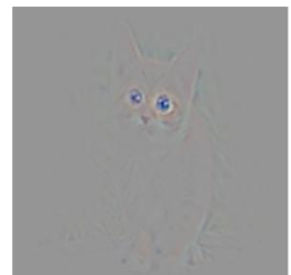
Backpropagation



Deconvolution



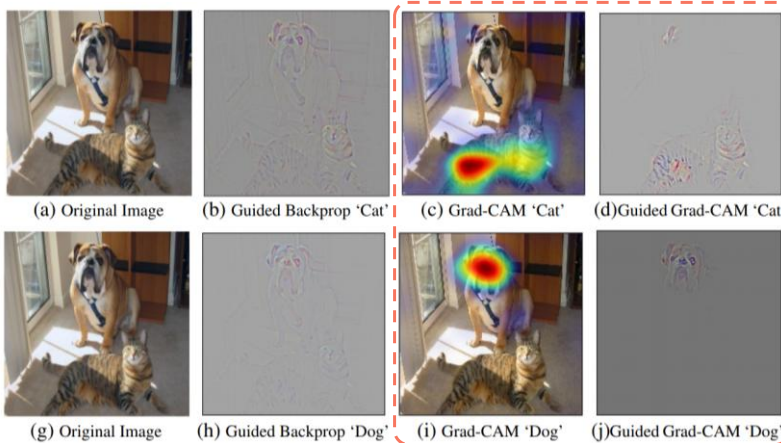
Guided Backprop



90

# A Saliency Map

- These pixel-space gradient visualizations (e.g., **DeconvNets** and **Guided Backpropagation**) are high-resolution and highlight fine-grained details in the image, but are not class-discriminative (i.e., fig. (b) and (h) are very similar).

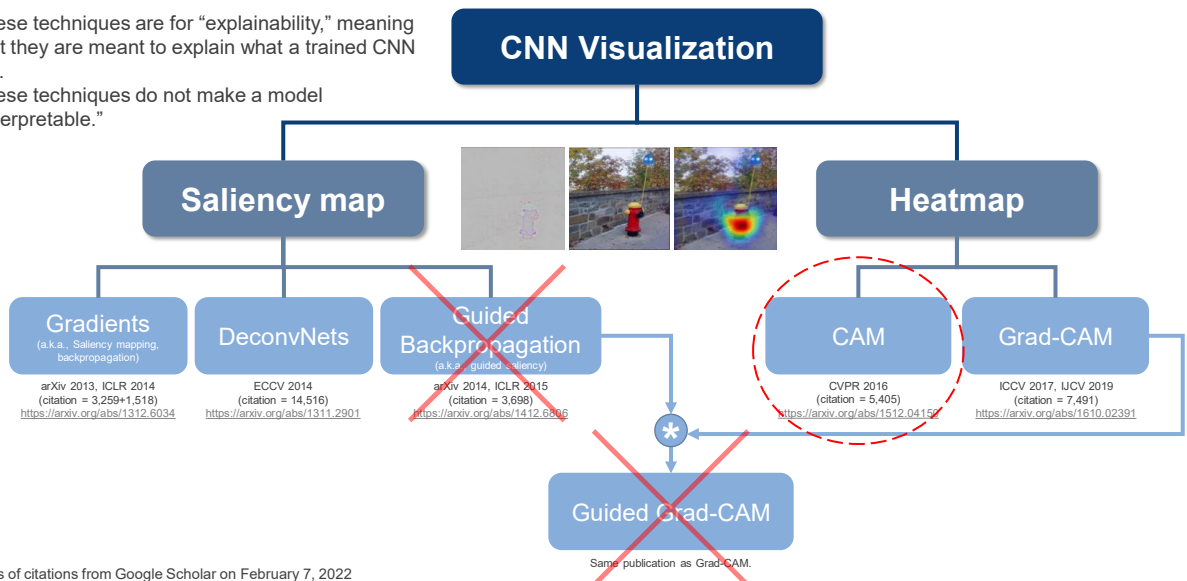


These visualizations are highly class-discriminative: the 'cat' explanation exclusively highlights the 'cat' regions but not 'dog' regions.

91

# A Visualizing where each class looks

- These techniques are for “explainability,” meaning that they are meant to explain what a trained CNN did.
- These techniques do not make a model “interpretable.”

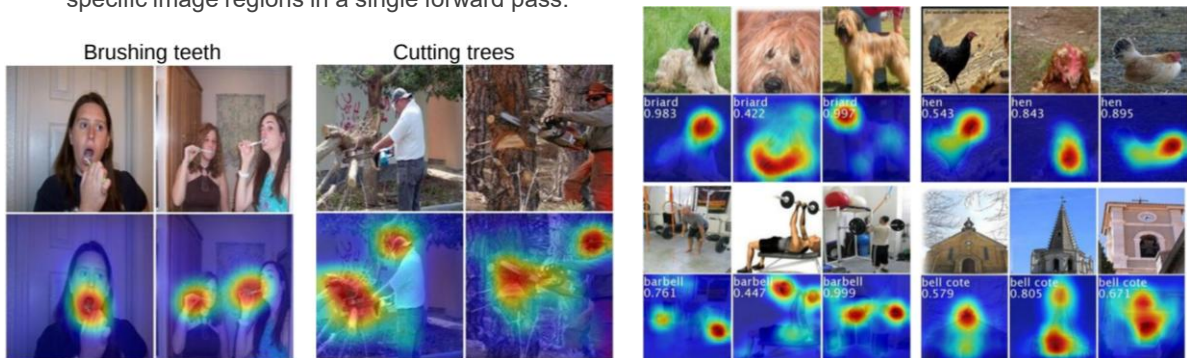


All numbers of citations from Google Scholar on February 7, 2022

92

# A Class Activation Map (CAM)

- **Class Activation Map (CAM)** was introduced in the paper entitled “Learning Deep Features for Discriminative Localization.” (MIT @CVPR2016) <https://arxiv.org/abs/1512.04150>
  - The output of **CAM** is “a class-discriminative localization map,” or a heatmap where the hot part corresponds to a discriminative region used by CNN to identify the output class. If there are  $n$  possible output classes, then for one input image, there can be  $n$  different **CAM** heatmaps as well.
  - This technique allows the classification-trained CNN to classify the image (as usual) and localize class-specific image regions in a single forward pass.



93

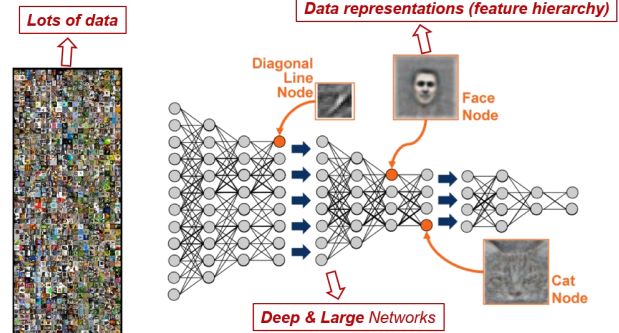
# A Class Activation Map (CAM)

- “Think of a deep network as a multistage information-distillation operation, where information goes through successive filters and comes out increasingly purified.”

– Francois Chollet, 2018.



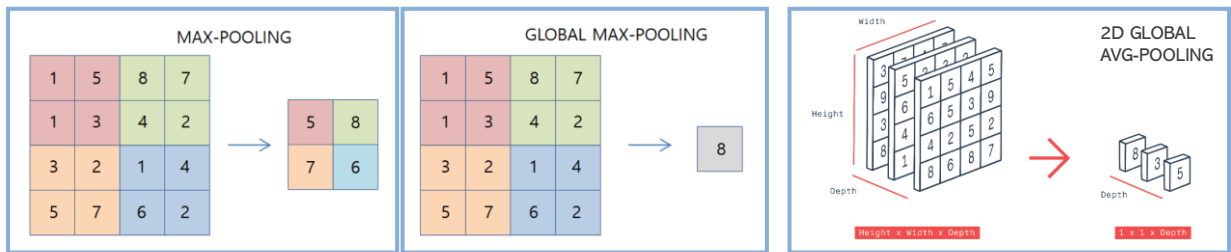
- Several previous works have asserted that deeper representations in a CNN capture the best high-level constructs.
- Also, CNNs naturally retain spatial information which is lost in fully-connected layers.
- Hence, we can expect the last convolutional layer to have the best tradeoff between high-level semantics and detailed spatial information.



94

# Global Pooling Layer

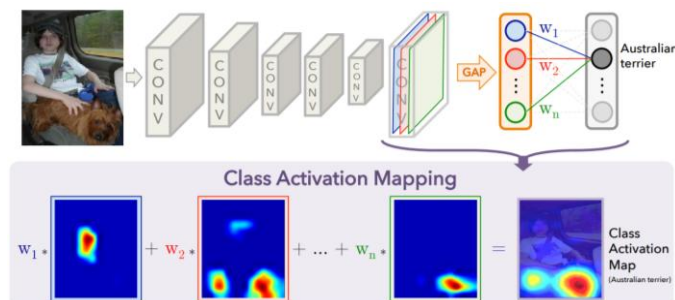
- **Global pooling** down samples the entire feature map to a single value.
- **Global pooling** can be used in a model to aggressively summarize the presence of a feature in an image.
- **Global pooling** is also sometimes used in models as an alternative to using a FC layer to transition from feature maps to an output prediction for the model. This is to minimize overfitting by reducing the total number of parameters in the model.
- In PyTorch, there is no layer for **global average pooling** and **global max pooling**, but both can be achieved by
  - Using adaptive pooling layers with an output size of 1, OR
  - Using standard pooling layers with the kernel size set to the dimensions of the input feature map.



95

## Class Activation Map (CAM)

- **CAM** is simply a weighted linear sum of the presence of visual patterns at different spatial locations.
  - **Global Average Pooling (GAP)** is preferred over Global Max Pooling layer (GMP) here. Because in **GAP**, we take an average across all the activation which helps to find all discriminative regions (whereas GMP layer just considers only the most discriminative one).
  - We can identify the importance of the image regions by projecting back the weights of the output layer on the convolutional feature maps obtained from the last convolutional layer.

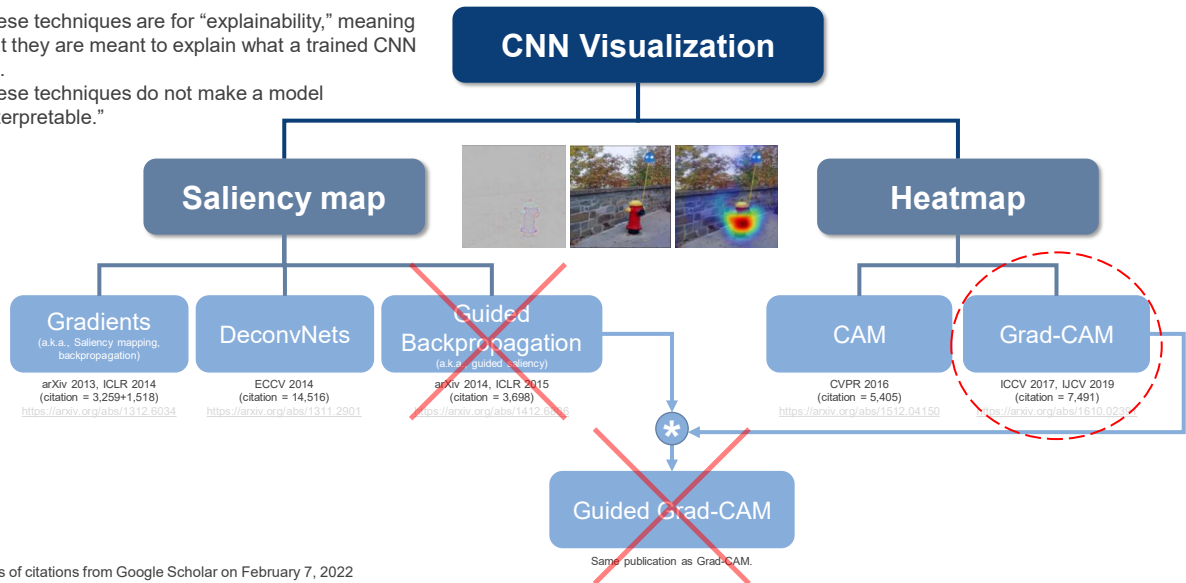


- Note that the same technique can be used for regression problems and other losses.

96

# A Visualizing where each class looks

- These techniques are for “explainability,” meaning that they are meant to explain what a trained CNN did.
- These techniques do not make a model “interpretable.”



All numbers of citations from Google Scholar on February 7, 2022

97

# A Gradient-weighted CAM (Grad-CAM)

- **Limitations of CAM:**
  - CAM requires feature maps to directly precede the softmax layers. Hence, it is only applicable to a particular kind of CNN architectures that perform global average pooling over convolutional maps immediately before prediction (i.e., conv feature maps > global average pooling > softmax layer).
  - This kind of restricted architecture may achieve in inferior accuracies compared to general networks on some tasks or simply be inapplicable to new tasks.
- **Gradient-weighted Class Activation Map (Grad-CAM)** (nickname: a gradient camera) was introduced in the paper entitled “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization.” (Georgia Institute of Technology @ICCV2017 @IJCV2019) <https://arxiv.org/abs/1610.02391>
- **Grad-CAM** can be seen as a generalization of CAM. In fully-convolutional architecture, **Grad-CAM** reduces to CAM.

98

# Gradient-weighted CAM (Grad-CAM)

## • CAM vs. Grad-CAM:

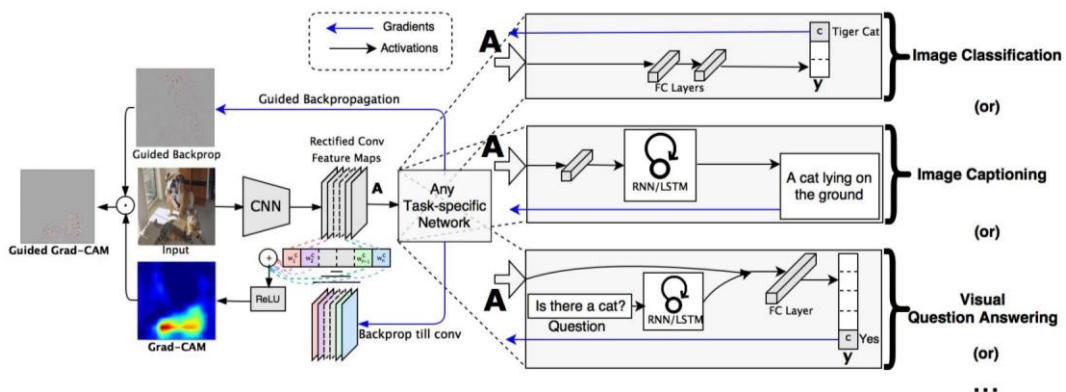
- Like CAM, the output of **Grad-CAM** is a class-discriminative localization map. So it can be used to visually debug where a network is looking in an image.
- Like CAM, **Grad-CAM** uses the feature maps produced by the last convolutional layer of a CNN.
- Unlike CAM, **Grad-CAM** is easily implemented, requires no modifications to the existing model architecture and needs no specific architecture.
  - In **Grad-CAM**, there can be any architecture following the last convolutional layer.
  - Grad-CAM** works with nearly any CNN-based architecture, including those for image captioning and visual question answering.
- Unlike CAM, **Grad-CAM** involves gradient computation. Hence, the only requirement in **Grad-CAM** is that layers following the last convolutional layer have to be differentiable so that gradients can be computed.

99

# Gradient-weighted CAM (Grad-CAM)

## • How Grad-CAM works:

- In our explanation, we assume the image classification task where:
  - $y_c$  corresponds to the raw class score output (i.e., the unnormalized predicted value before Softmax) of the network regarding the target class  $c$ .
  - Suppose that there are  $k$  2D feature maps produced after the last convolutional layer, then  $A_i$  refers to the  $i^{th}$  feature map activation (height= $v$ , width= $u$ ) when  $i \in \{1, 2, 3, \dots, k\}$ .



100

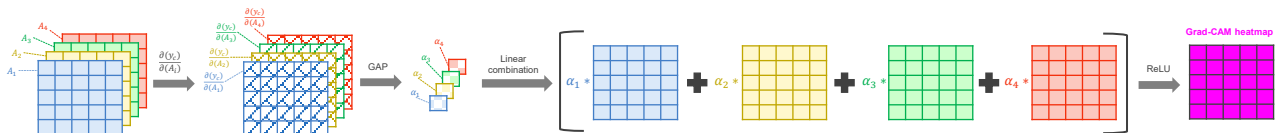
# Gradient-weighted CAM (Grad-CAM)

## How Grad-CAM works:

1. Compute the gradient of  $y_c$  with respect to each feature map activation  $A_i$ ; this means computing  $\frac{\partial(y_c)}{\partial(A_i)}$  for all  $i \in \{1, 2, 3, \dots, k\}$ 
  - For a 2D input image, the resultant gradient of this step has a 3D shape of  $(k, v, u)$ .
  - The resultant gradient can be seen as a stack of  $k$  maps where each map is a 2D gradient (shape= $(v, u)$ ) of  $y_c$  with respect to a single 2D feature map  $A_i$ .
2. Compute a vector of  $k$  alpha values or  $\{\alpha_1, \alpha_2, \alpha_3, \dots, \alpha_k\}$ , where  $\alpha_i$  is a scalar obtained by global average pooling the 2D gradient (computed regarding  $A_i$ ).
3. Compute the final Grad-CAM heatmap (shape= $(v, u)$ ) by

$$\text{Heatmap} = \text{ReLU}(\alpha_1 A_1 + \alpha_2 A_2 + \dots + \alpha_k A_k)$$

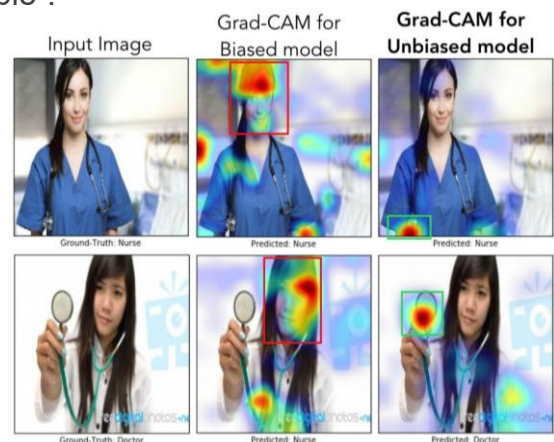
ReLU is applied to emphasize only the positive gradient values and turn all the negative gradient values into zero.



101

# Gradient-weighted CAM (Grad-CAM)

- **Grad-CAM** is a technique for “explainability”—it is meant to explain what a trained CNN did.
- **Grad-CAM** does not make a model “interpretable”.
- While **Grad-CAM** heatmaps often make sense, they aren’t required to make sense.
- **Grad-CAM** is useful for:
  - Weakly-supervised localization or weakly supervised segmentation
  - Debugging a model and gaining more insight into why the model is making certain mistakes

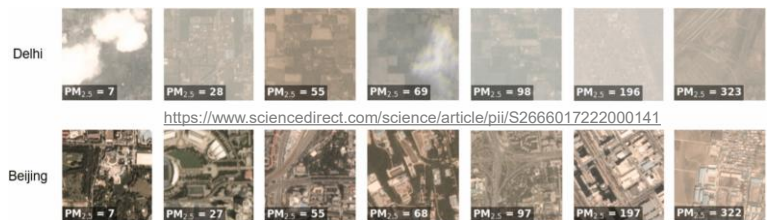


102

# A Gradient-weighted CAM (Grad-CAM)

## • Grad-CAM IS NOT quite useful when:

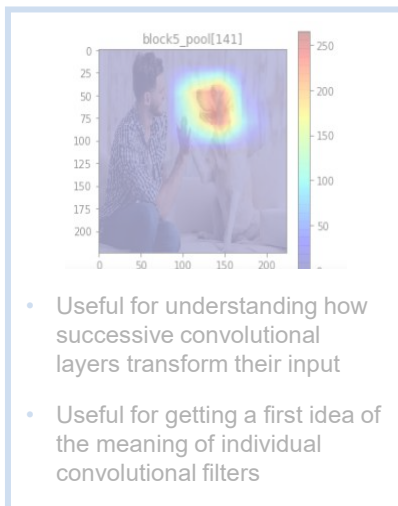
- **Multiple Instances of Same Class:** Difficult to differentiate individual objects when many of the same class are present.
- **Small or Fine-Grained Objects:** Coarse resolution of heatmaps limits detailed explanation for tiny or intricate features.
- **Spurious Correlations:** May highlight irrelevant background elements the model falsely associated with the class.
- **Noisy or Heavily Occluded Images:** Unreliable results due to compromised gradient signals.
- **Pixel-Level Granularity Needed:** Not ideal for explaining at the exact pixel level.
- **Global Context Decisions:** Less effective when the model relies on overall image context, not just local features.
- **Precise Multi-Class Localization:** Challenges in clearly distinguishing heatmaps for closely interacting multiple classes.



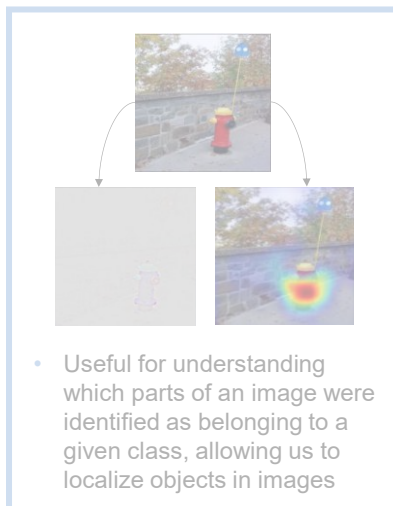
103

# A Visualizing what CNN learns

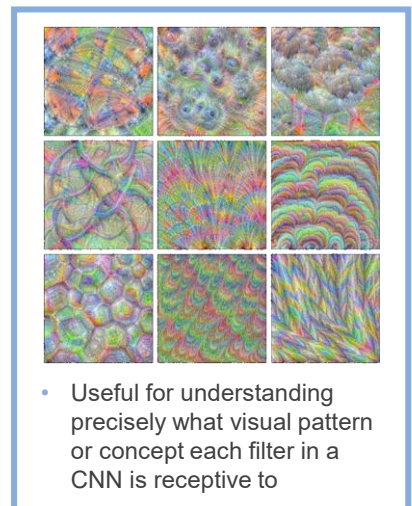
## INTERMEDIATE ACTIVATIONS



## WHERE EACH CLASS LOOKS



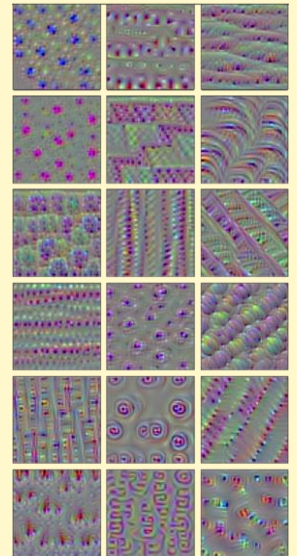
## CNN FILTERS



104

# Visualizing CNN filters

- Drawbacks of gradient-based methods:
  - Gradient-based approaches are attractive in their simplicity, but the optimization process tends to produce images that do not greatly resemble natural images.
  - Instead, they are composed of a collection of “hacks” that happen to cause high (or low) activations: extreme pixel values, structured high-frequency patterns, and copies of common motifs without global structure.
- If we want to know what kind of pattern significantly activates a certain feature map, there are two main ways to try:
  1. Try to find images in a dataset that result in a high average activation of this feature map, **OR**
  2. Try to generate such a pattern by optimizing the pixel values in a random image.
    - This method was first proposed in 2009. D. Erhan, Y. Bengio, A.C. Courville, and P. Vincent, “Visualizing higher-layer features of a deep network,” Technical Report, University of Montreal, 1341(3), 2009 [https://www.researchgate.net/publication/265022827\\_Visualizing\\_Higher-Layer\\_Features\\_of\\_a\\_Deep\\_Network](https://www.researchgate.net/publication/265022827_Visualizing_Higher-Layer_Features_of_a_Deep_Network)



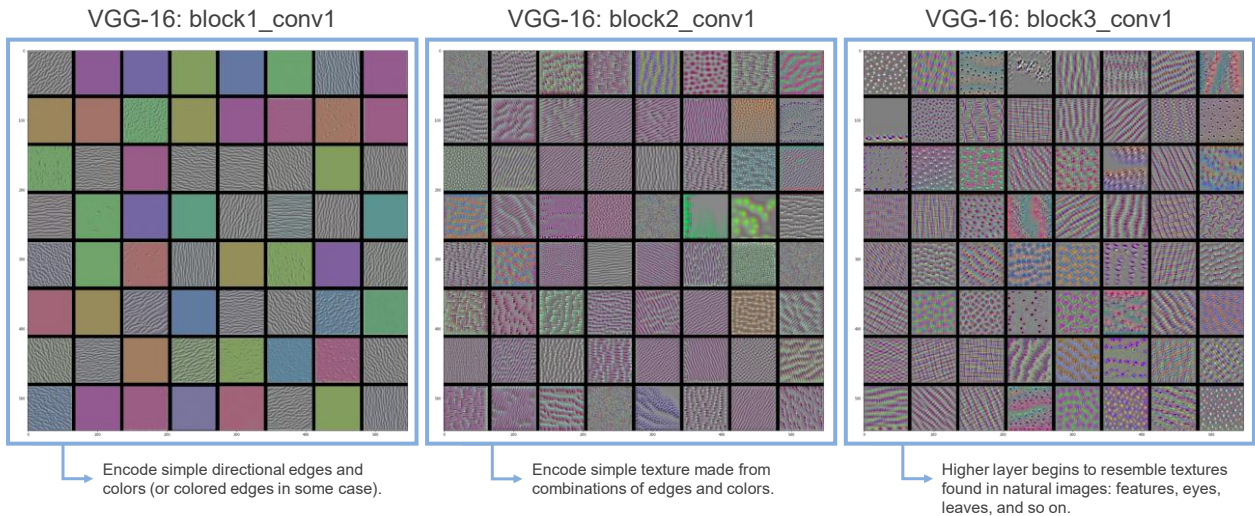
105

# Visualizing CNN filters

- We try to find an image that maximizes the average activation of a certain feature map by performing gradient descent optimization on the pixel values.
- **Step-by-Step:**
  1. Load the pre-trained network (*model*), for example, VGG-16 pre-trained on ImageNet.
  2. Prepare an initial image (*I*), which can be an empty image or just a random-noise image.
  3. Update all pixels in *I*:
    - 1) Do forward passing using *I* as an input image to *model*
    - 2) Trap outputs from the target convolutional layer (*layer\_output*); these outputs consist of multiple 2D activation maps regarding all filters in this convolutional layer.
    - 3) From *layer\_output*, choose only the activation map regarding the one target filter (*filter\_output*)
    - 4) As *filter\_output* is a single 2D activation map, we compute *loss* that is the mean of all pixels in *filter\_output*.
    - 5) Find gradients of *loss* with respect to *I*, or  $grads = \partial(loss)/\partial(I)$ .
    - 6) Updating *I* by **Gradient Ascent** using  $I += grads$ .
  4. Repeat 3 for a number of iterations (e.g., 40 iterations per one convolutional filter) until most results seem to converge.

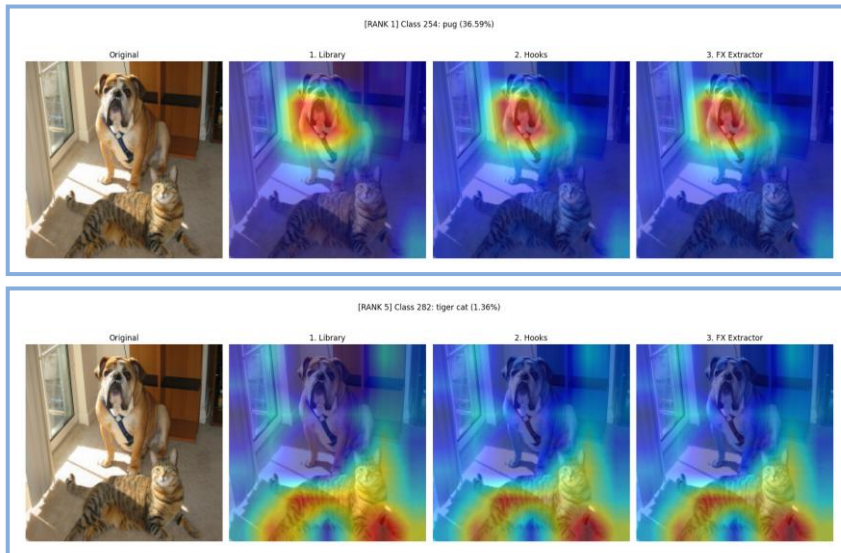
106

# Visualizing CNN filters



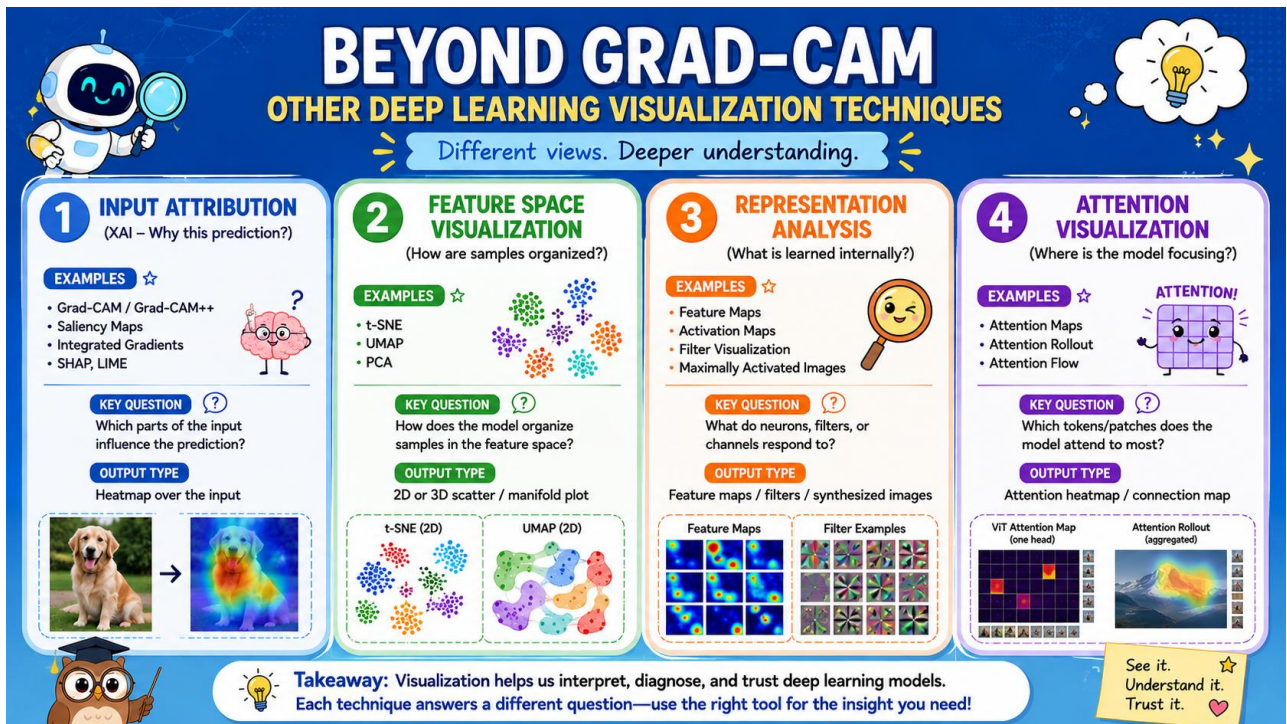
107

## EX4: Visualizing CNN's Grad-CAM



- **What to understand or explain?**  
A prediction? A class? An intermediate representation? Attention? Failure cases?
- **Which visualization/ interpretability technique suits this architecture and goal?**  
Grad-CAM, activation maps, saliency, attention visualization, feature visualization, etc.
- **Which layer should be targeted for visualization?**  
Earlier vs. later layers can reveal very different information.
- **What exactly does the visualization represent?**  
For example, Grad-CAM shows spatial regions associated with increasing a particular class score—not simply “where the model looks.”
- **How to validate the interpretation?**  
Does the visualization behave sensibly across different images/classes? Does it change when the prediction changes? Is the apparent explanation actually supported by the method?

108



109

## EX4: Visualizing CNN's Grad-CAM

- Three PyTorch alternatives to Grad-CAM:

### 1. Instant libraries:

- TorchCAM
- Captum by Meta (the official PyTorch interpretability library)
- pytorch-grad-cam (<https://github.com/jacobgil/pytorch-grad-cam>)
- ...

### 2. The classic forward and backward hooks:

- `target_layer.register_forward_hook()` for capturing feature (activation) maps
- `target_layer.register_full_backward_hook()` for capturing gradients w.r.t. the feature (activation) maps

Work on any model. Only fail when the specific target layer is a functional op (no `nn.Module` to attach to) or an in-place op (input/output collapse into one tensor). Hooking a containing module is unaffected either way.

(Optional) Can modify activations (fwd hook) or gradients (bwd hook) as they pass through.



110

# EX4: Visualizing CNN's Grad-CAM

## • Three PyTorch alternatives to Grad-CAM: (cont.)

FX — PyTorch's toolkit for tracing and transforming models into an intermediate graph representation

### 3. The newer, cleaner, and more stable FX-based extraction:

- `torchvision.models.feature_extraction.create_feature_extractor()` instead of a forward hook
- `activation_maps.retain_grad()` on the (non-leaf) activation tensor instead of a backward hook, to preserve its gradient
- `score.backward()` to trigger the backward computation

Handles functional ops and in-place ops cleanly, since tracing turns them into explicit graph nodes — no attachment-point problem.

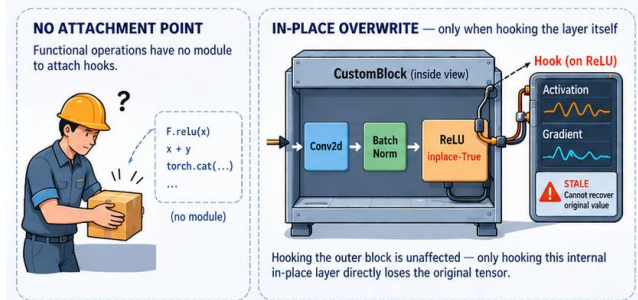
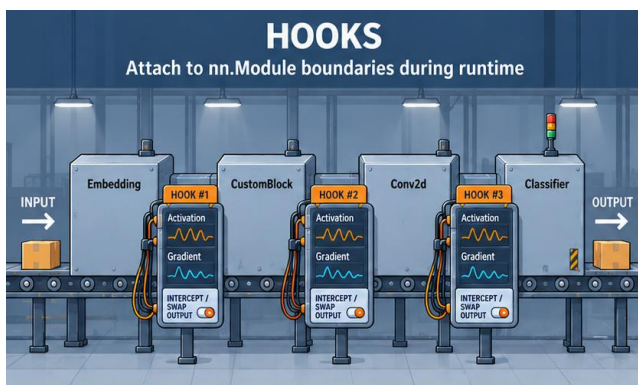
Read-only: cannot intervene in the computation itself.

**BUT** the model must be traceable in the first place; dynamic control flow or Python operations in `forward()` that require a concrete tensor value at trace time (e.g., `if x.sum() > 0: x.item()`, tensor-length-dependent loops) — FX can't resolve these on a symbolic placeholder.

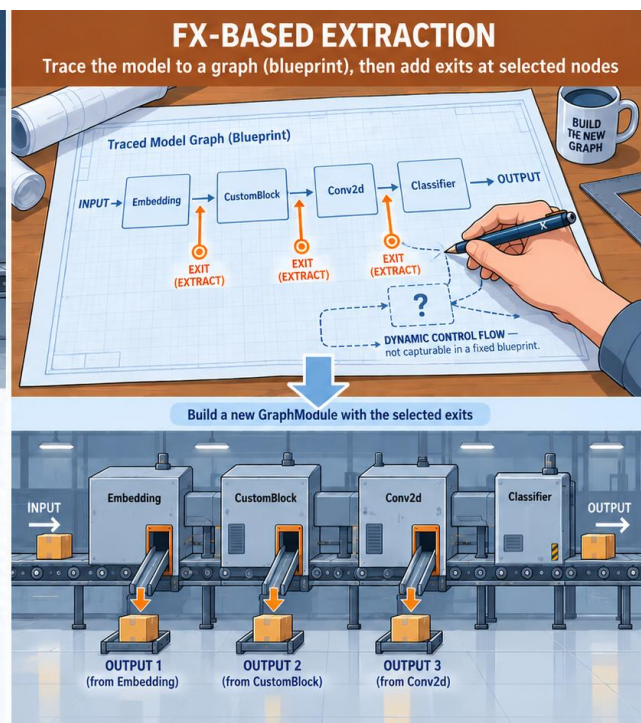
In any implementation, **DON'T** wrap Grad-CAM forward in `torch.no_grad()` or `torch.inference_mode()`. Otherwise, backward will produce no gradients.

**A**

111



112





|                                                     | Hooks                                                                                                                            | FX-based Extraction                                                                                                        |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Touches original model?</b>                      | No structural rewrite — registers callbacks on existing modules                                                                  | No in-place rewrite — traces model and returns a new <code>GraphModule</code>                                              |
| <b>Works with dynamic models?</b>                   | ✔ Yes — observes actual runtime execution                                                                                        | ✗ Only if symbolically traceable; data-dependent control flow can fail                                                     |
| <b>Can capture</b>                                  | <code>nn.Module</code> calls, e.g. <code>nn.ReLU</code> , <code>nn.Conv2d</code>                                                 | ✔ Module + functional + method operations, once successfully traced                                                        |
| <b>Cannot directly capture / problems with</b>      | ✗ Functional ops such as <code>F.relu(x)</code> — no module to attach to<br>⚠ In-place ops can complicate capture/backward hooks | ✗ Data-dependent control flow, e.g. <code>if x.sum() &gt; 0:</code><br>✗ Some non-traceable Python/custom/third-party code |
| <b>Can modify activations / gradients mid-flow?</b> | ✔ Yes — pre-hooks, forward hooks, full backward hooks                                                                            | <code>create_feature_extractor()</code> : intended for feature extraction, not runtime intervention                        |
| <b>Best used when...</b>                            | You need runtime inspection/intervention, gradients, or dynamic/non-traceable execution                                          | You need clean, declarative extraction of intermediate nodes from a traceable model                                        |



113



# Homework

114

# Image: Extend the pre-trained CNN

- **Objective:** What does it take to train an image classifier with our custom image dataset?
  - Data collection
  - Data preparation
  - Architecture the model
    - Backbone selection
    - Finetuning strategy
    - Hyperparameter tuning
  - Training and logging
  - Evaluate and visualize
    - Quantitatively: All with mean  $\pm$  SD
    - Qualitatively: Grad-CAM
    - Qualitatively: good cases vs. bad cases
  - Ablation study



A

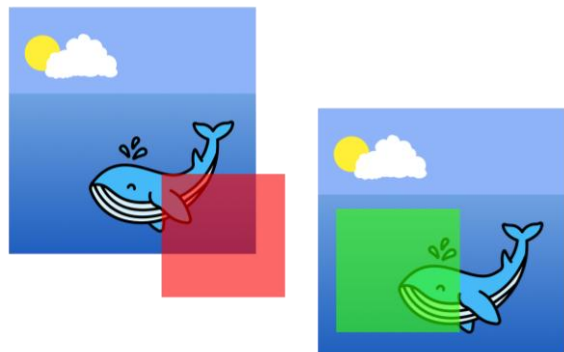
115

## Unmatched image dimension, what to do?

Please refer to the explanation in Section 2.2 of CNN2\_ex3.ipynb



Resize



Random crop

RandomResizedCrop

<https://pytorch.org/vision/main/generated/torchvision.transforms.RandomResizedCrop.html>

A

116

# A MEMORY REQUIREMENT

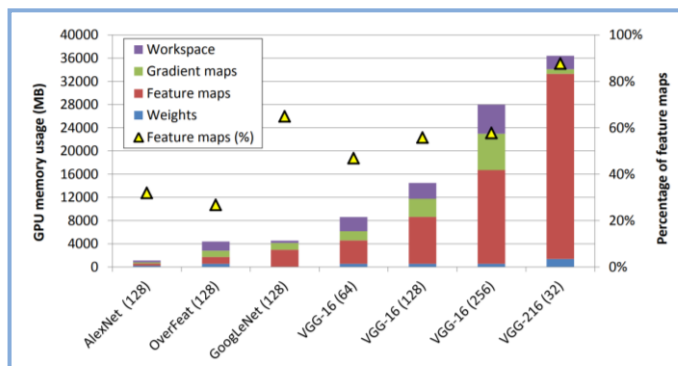
- During inference, the GPU RAM occupied by one layer can be released as soon as the next layer has been computed. Hence, we only need as much GPU RAM as required by two consecutive layers.
- **BUT** during training, everything computed during the forward pass needs to be preserved for the reverse pass, so the amount of GPU RAM needed is (at least) the total amount of RAM required by all layers.



117

# A Anatomy of Model's Memory

- Apart from the model itself, there are many components during training that use GPU memory. (Ref: [https://huggingface.co/docs/transformers/perf\\_train\\_gpu\\_one](https://huggingface.co/docs/transformers/perf_train_gpu_one))
  - Model weights
  - Optimizer states
  - Gradients
  - Forward activations saved for gradient computation
  - Temporary buffers
  - Functionality-specific memory
- For CNN, feature maps consume the most GPU memory.



NVIDIA, "vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design," MICRO, 2016. <https://arxiv.org/abs/1602.08124>

118

# A OUT-OF-MEMORY ERROR

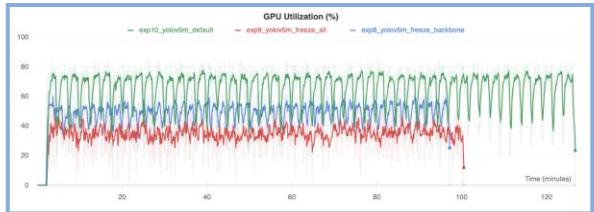
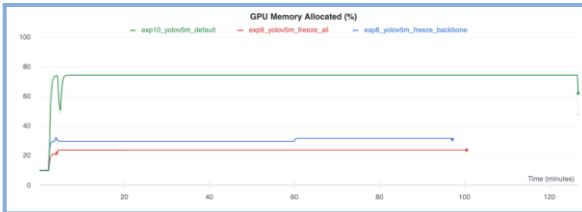
- Use another GPU with bigger memory
- Train on multiple GPUs (for example, model parallelism, data parallelism)
- Use smaller data (for example, smaller batch size, smaller image size)
- Use a smaller model (for example, remove some layers)
- Use a model architecture that consumes less GPU memory (for example, use pooling or strided convolution to reduce the dimension of intermediate feature maps)
- Reduce memory usage for parameters and computed values (for example, FP16 training mode or the mixed-precision training mode, gradient checkpoint)
- Don't load the whole training dataset to GPU RAM



119

# A OUT-OF-MEMORY ERROR

- Does freezing some layers help lower GPU memory usage?



Credit: [https://docs.ultralytics.com/yolov5/tutorials/transfer\\_learning\\_with\\_frozen\\_layers/#gpu-utilization-comparison](https://docs.ultralytics.com/yolov5/tutorials/transfer_learning_with_frozen_layers/#gpu-utilization-comparison)

- Other techniques to reduce GPU memory when training on a single GPU:

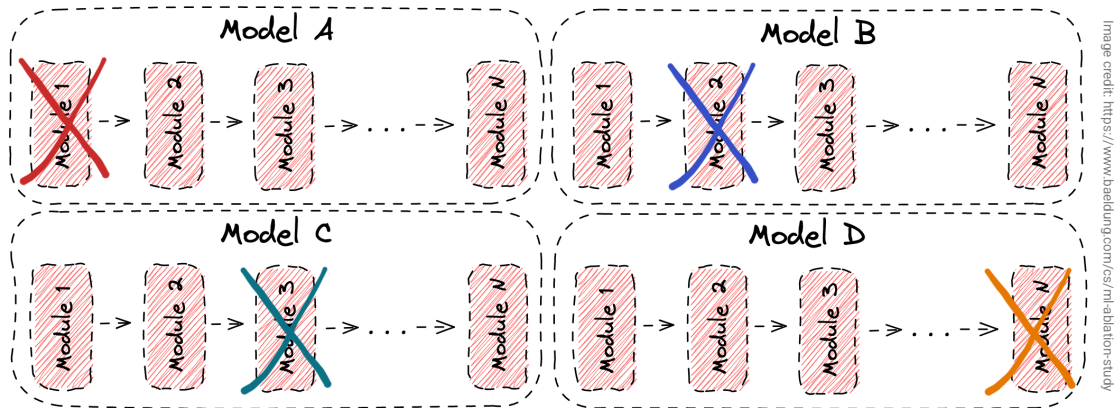
(Ref: [https://huggingface.co/docs/transformers/perf\\_train\\_gpu\\_one](https://huggingface.co/docs/transformers/perf_train_gpu_one))

| Method                   | Improve speed | Improve memory |
|--------------------------|---------------|----------------|
| Gradient accumulation    | No            | Yes            |
| Gradient checkpointing   | No            | Yes            |
| Mixed precision training | Yes           | (No)           |
| Batch size               | Yes           | Yes            |
| Optimizer choice         | Yes           | Yes            |
| DataLoader               | Yes           | No             |
| DeepSpeed Zero           | No            | Yes            |

120

# Ablation Study

- **An ablation study** comes where certain parts of the network are removed in order to gain a better understanding of the network's behavior.



Each time we remove one of the modules and check the performance of the new model to investigate the influence of the removed module.

121



122